

DEEP STUDY

Docker Mastery Syllabus

سیلابس جامع تسلط عمیق بر Docker و پشته‌ی کانتینر

Linux Containers / OCI / containerd / runc / Implementation

1.1.0	نسخه
Canonical Foundation	وضعیت
2026-08-11	Research cutoff
Master Curriculum / Pre-first-module baseline	نقش سند
Vazirmatn	فونت اصلی

این سند نقشه‌ی اصلی مسیر Docker در پروژه Deep Study است. نسخه‌ی Markdown منبع اصلی و قابل ویرایش است و PDF برای مطالعه استفاده می‌شود.

فهرست مطالب

7	هدف سند
7	اصل بنیادی: PDF خروجی مطالعه است، نه Source of Truth
7	قرارداد یکپارچگی PDF و Knowledge Base
8	Baseline پژوهش این نسخه
8	هر Module چه خروجی‌ای دارد؟
9	نردبان تسلط
9	Dependency chains کلیدی
9	فهرست فازها و مازول‌ها
13	P0 - قرارداد پروژه، روش مطالعه و آزمایشگاه
13	META.DKR.01 - قرارداد تسلط Docker و Scope کل برنامه
14	META.DKR.02 - محیط آزمایش و ابزارهای مشاهده سیستم
15	META.DKR.03 - سیستم Coverage و Snapshot کامل Reference
15	P0 Gate
16	P1 - Docker به عنوان محصول و runtime روزمره
16	DKR.01 - تاریخچه و مدل ذهنی Container
17	DKR.02 - نصب، Platforms، Editions، Distribution
18	DKR.03 - معماری runtime stack -> dockerd -> API -> CLI Docker Engine
19	DKR.04 - Docker Object Model و Inspectability
20	DKR.05 - Container Lifecycle کامل
21	DKR.06 - Process Model، PID 1، Signals، TTY و I/O
22	DKR.07 - Resource Constraints، OOM، PIDs، Devices و Accelerators
23	DKR.08 - Filesystem view و Container Mutation Operations
23	P1 Gate
24	P2 - Images، Dockerfile، BuildKit و Distribution
24	DKR.09 - Image Mental Model: layers, config, manifest, digest
25	DKR.10 - Naming, Tags, Digests, Content Addressability و Image Lifecycle
25	DKR.11 - Dockerfile: Semantics کامل دستورات
26	DKR.12 - Build Context، .dockerignore و Context Types
27	DKR.13 - Layering، Cache و Reproducible Builds
28	DKR.14 - BuildKit Architecture و LLB
29	DKR.15 - Buildx، Builder Instances، Drivers و Bake

29	Cross Building و DKR.16 - Multi-platform Images
30	Entitlements و DKR.17 - Advanced Build Mounts, Secrets, SSH, Devices
31	Provenance Attestations و DKR.18 - Exporters, Cache Backends, SBOM
32	Authentication و DKR.19 - Registries, OCI Distribution, Docker Hub
33	P2 Gate
34	Linux boundary و Docker و P3 - Storage
34	DKR.20 - Persistence Model: Volumes, Bind Mounts, tmpfs
35	Overlay Concepts و DKR.21 - Storage Drivers, Copy-on-Write
36	Docker Engine و Snapshotters و DKR.22 - containerd Image Store, Content Store
37	Network Drivers و DKR.23 - Docker Networking Model
38	DKR.24 - Bridge Networking Deep Dive: veth -> bridge -> routing -> NAT
39	DKR.25 - Advanced Network Drivers: host/none/macvlan/ipvlan/IPv6
39	Swarm Data Plane و DKR.26 - Overlay Networking
40	DKR.27 - Networking Troubleshooting Playbook
41	P3 Gate
42	Swarm و P4 - Compose, Engine Administration, Observability, Security
42	Specification و DKR.28 - Compose Application Model
43	Reuse و DKR.29 - Compose Advanced: Merge, Include, Profiles, Watch
44	Host Integration و DKR.30 - dockerd Administration
45	Upgrades و DKR.31 - Versioning, API Compatibility, Deprecations
46	Diagnostics و DKR.32 - Logs, Events, Metrics
47	DKR.33 - Container/Daemon Security Model
48	Rootless Docker و DKR.34 - User Namespaces, usersns-remap
49	Software Supply Chain Security و DKR.35 - Image
50	Trust Boundaries و DKR.36 - Credentials, Secrets
51	PKI و DKR.37 - Swarm Architecture: Nodes, Services, Tasks, Raft
51	DR و DKR.38 - Swarm Operations: Overlay, Routing Mesh, Updates, Secrets
52	Extension APIs و DKR.39 - Docker Engine Plugin System
53	P4 Gate
54	Container و P5 - Linux internals
54	file descriptors و LNX.01 - Linux Process Model, syscalls, /proc
54	API و LNX.02 - Linux Namespaces: همه انواع و
55	ID Mapping Internals و LNX.03 - User Namespace
56	delegation و LNX.04 - cgroup v2: hierarchy, controllers

57	 pivot_root و LNX.05 - Mounts, VFS, rootfs, chroot
57	 LNX.06 - OverlayFS Internals
58	 Containers برای LNX.07 - Linux Networking Fundamentals
59	 nftables/iptables و LNX.08 - veth, Linux Bridge, NAT, conntrack
60	 devices و LNX.09 - Linux Security: capabilities, seccomp, LSM
60	 Container Host Integration و LNX.10 - systemd, cgroups
61	 P5 Gate
62	 runc و P6 - OCI, containerd
62	 OCI.01 - OCI Image Specification
62	 OCI.02 - OCI Runtime Specification
63	 OCI.03 - OCI Distribution Specification
64	 Non-image Content و OCI.04 - OCI Artifacts, Referrers
65	 Plugin/Service Model و CTR.01 - containerd Architecture
65	 GC و CTR.02 - containerd Content Flow, Images, Snapshots, Leases
66	 Shim و CTR.03 - containerd Containers vs Tasks, Runtime v2
67	 Runtime Lifecycle و RNC.01 - runc CLI, OCI Bundle
68	 CRIU و RNC.02 - libcontainer Internals, Bootstrap/Init
68	 P6 Gate
69	 P7 - Source Code Reading و ساخت اجزای مشابه
69	 Test و SRC.MOB.01 - Moby Repository Architecture, Build
69	 Moby کردن Code Paths واقعی در Moby SRC.MOB.02 - Trace
70	 API Mapping و SRC.CLI.01 - Docker CLI Source
71	 Frontend و SRC.BLD.01 - BuildKit Source: Solver, Worker, Cache, Executor
71	 Compose Implementation Source و SRC.CMP.01 - Compose Specification
72	 Mini Container Runtime ساخت - LAB.01
73	 Mini OCI Image/Registry Explorer ساخت - LAB.02
73	 Docker بدون Docker Bridge Networking بازسازی - LAB.03
74	 P7 Gate
75	 P8 - Cross-platform, Docker Product Ecosystem و قابلیت‌های جدید
75	 Host Integration و PLT.01 - Docker Desktop Architecture
76	 Windows Containers و PLT.02 - Windows: WSL2, Hyper-V
76	 Product Surface به‌عنوان Hardened Images و PLT.03 - Docker Hub, Scout
77	 PLT.04 - Emerging Docker AI/Agent Surface: Model Runner, Sandboxes, MCP Toolkit
78	 P8 Gate

79	Interview و P9 - Production, Failure Engineering, Ecosystem, Contribution
79	Lifecycle Management و OPS.01 - Production Engineering
80	OPS.02 - Performance Engineering
80	Troubleshooting Matrix و OPS.03 - Failure Engineering
81	Boundary Reasoning و OPS.04 - Ecosystem Comparison
82	Merged PR تا Issue: CONTRIB.01 - Contribution Workflow
82	INT.01 - Interview Mastery: 30 seconds / 3 minutes / 30 minutes
83	Production Scenario Drills و INT.02 - Adversarial Interview
84	Re-certification و CAP.01 - Final Docker Mastery Capstone
84	P9 Gate
85	قرارداد Master Prompt و Visual QA برای همه PDFها
86	استاندارد PDFهای بعدی
87	روش سیستماتیک استخراج Syllabus از منابع معتبر
87	1. Canonical documentation inventory
87	2. Book TOC extraction
87	3. Standards/specifications
87	4. Reference surface audit
87	5. Source-tree audit
87	6. Dependency extraction
87	7. Release/deprecation audit
88	8. Union + normalization
88	9. Classification
88	10. Dependency DAG -> Spiral Order
88	11. Coverage Matrix
88	12. Adversarial gap review
89	Protocol مطالعه دوفره
90	ساختار پیشنهادی Repository
91	Source Policy و کتابها
92	Coverage Matrix Schema
93	Mastery Checklist عمومی
94	Prompt pattern برای استخراج/بهروزرسانی Syllabus
95	Bibliography / Canonical Source Registry

97	Change Policy
98	Changelog - v1.1.0

هدف سند

- این سند ستون فقرات مسیر Deep Study برای Docker است. هدف آن «تمام کردن Docker» نیست؛ هدف ساختن مدلی است که از usage تا implementation و contribution امتداد دارد.
- هر module در آینده می‌تواند به یک Study PDF مستقل (و در صورت نیاز Workbook جدا) تبدیل شود. هیچ PDF نباید بدون Scope Contract، منابع canonical و Definition of Done ساخته شود.
- این نسخه آگاهانه Docker را در دو سطح می‌بیند: (۱) container/Engine/open-source stack که باید بسیار عمیق شود؛ (۲) product ecosystem شرکت Docker که باید گسترده و version-sensitive شناخته شود.

اصل بنیادی: PDF خروجی مطالعه است، نه Source of Truth

- در repository، نسخه Markdown/structured text منبع قابل version-control است و PDF artifact نهایی مطالعه. اصلاحات ابتدا در source انجام می‌شوند و سپس PDF دوباره ساخته می‌شود.
- برای هر PDF metadata شامل Document ID، version، last reviewed، research cutoff، scope، source baseline و prerequisites ثبت می‌شود.
- PDF باید synthesis باشد: + concepts + internals + source/spec provenance + labs + failure modes + documentation. assessment؛ نه بازنویسی طولانی

قرارداد یکپارچگی PDF و Knowledge Base

- فونت canonical همه PDFهای فارسی پروژه **Vazirmatn** است. Body، headings، tables، boxes، footer/page numbers و اعداد فارسی باید با Vazirmatn تولید شوند؛ وجود واقعی و embedding فونت با PDF preflight کنترل می‌شود.
- Master prompt پروژه در meta/prompts/MASTER_PDF_PROMPT.md قرار دارد و برای هر PDF آینده mandatory است. prompt اصلی کاربر بدون حذف در repository نگهداری می‌شود و master prompt لایه‌های Deep Study را به آن اضافه می‌کند.
- هر Study PDF یک Scope Contract دارد: In Scope، Out of Scope، Prerequisites، Assumed Knowledge، Required Mastery و Upstream/Downstream Dependencies. Version-sensitive Areas، Source Baseline Evidence.
- قبل از نوشتن هر syllabus، PDF فعلی + terminology registry + owner/upstream/downstream documents + source policy باید consistency-audit شوند.
- بعد از تولید preflight، PDF ماشینی کافی نیست: PDF باید به تصاویر render شود و تمام صفحات از نظر RTL/LTR، فونت، glyph، cover، TOC، borders، page numbers، headings، tables، boxes، code blocks، citations، page breaks بررسی شوند. هر defect باید در source اصلاح و PDF از نو ساخته/بازبینی شود.
- هیچ PDF با canonical status بدون ثبت Visual QA status/date و matching source version معتبر نیست.

Baseline پژوهش این نسخه

- 11-08-2026: Research cutoff. قبل از اجرای هر module version-sensitive باید canonical page و release notes فعلی دوباره باز و verify شوند؛ search snippets به‌تنهایی معیار «latest» نیستند.
- Docker Engine: شاخه 29؛ آخرین release مشاهده و مستقیماً روی صفحه رسمی release notes بررسی‌شده در این سند 29.7.2 (2026-08-05) است. BuildKit 29.7.2 را به v0.32.2 به‌روزرسانی می‌کند؛ release 29.7.0 نیز containerd v2.3.3 و runc v1.4.3 را برای static binaries گزارش می‌کند. component baseline باید هنگام هر module دوباره با packaging واقعی همان environment تطبیق داده شود.
- Docker Engine 29.0+ در fresh installations از containerd image store به‌عنوان storage backend پیش‌فرض استفاده می‌کند؛ upgrade‌های قدیمی ممکن است همچنان classic graph driver داشته باشند.
- OCI snapshot بررسی‌شده: Runtime Spec v1.3.0؛ Image Spec v1.1.1؛ Distribution Spec v1.1.1. این نسخه‌ها برای pin کردن reading استفاده می‌شوند ولی syllabus به یک version محدود نیست.

Deltaهای مهم current baseline که باید در moduleهای مربوط دیده شوند

- Engine 29.0+ در fresh install به containerd image store مهاجرت کرده؛ تفاوت upgrade install و userns-remap caveat در DKR.22/DKR.31 بررسی می‌شود.
- Engine 29.7.0 یک feature آزمایشی embedded-containerd اضافه کرده که topology سنتی dockerd -> external managed containerd را optional می‌کند؛ DKR.03/DKR.22 باید logical contract را از deployment topology جدا آموزش دهند.
- image mount type در 29.7.0 از experimental خارج شده و باید در storage/mount coverage ثبت شود.
- default-stop-timeout در 29.7.0 به daemon configuration اضافه شده؛ lifecycle/daemon administration باید version caveat آن را بشناسد.
- cgroup v1 در Engine 29 deprecated است؛ مسیر عمیق پروژه بر cgroup v2 متمرکز است و legacy فقط برای migration/debug awareness نگه‌داری می‌شود.
- rootless networking defaults در شاخه 29 تغییر کرده‌اند؛ هر rootless module باید implementation/default واقعی release مورد مطالعه را دوباره verify کند.
- از Docker v29، Go module قدیمی github.com/docker/docker deprecated است؛ source/contributor path باید public modules جدید Moby client/api و root module binary-oriented را بشناسد.

هر Module چه خروجی‌ای دارد؟

- Core Study PDF: متن آموزشی canonical برای scope همان module.
- Scope Contract: In Scope / Out of Scope / Prerequisites / Downstream Dependencies / Version-sensitive areas.
- Labs: آزمایش‌های سالم و experiments برای مشاهده mechanism.
- Failure Pack: خطاهای عمدی و debugging checklist.
- Source/Spec Reading Tasks: دقیقاً کدام specification section یا code path باید خوانده شود.
- mastery checklist و Assessment: teach-back، interview ladder، diagrams-from-memory.

نردبان تسلط

- **User - L1**: عملیات اصلی را انجام می‌دهد و terminology را می‌شناسد.
- **Power User - L2**: reference را سریع پیدا می‌کند، edge cases و configuration surface را می‌شناسد.
- **Production Operator - L3**: امن و قابل‌اعتماد deploy/upgrade/observe می‌کند و failureهای رایج را حل می‌کند.
- **Troubleshooter/Architect - L4**: مشکل را cross-layer debug می‌کند و architectureهای trade-off را توضیح می‌دهد.
- **Internals Expert - L5**: Docker abstraction را تا OCI/containerd/runc/Linux primitives می‌شکند و source را trace می‌کند.
- **Contributor - L6**: issue واقعی را reproduce، test/patch و در review فنی مشارکت می‌کند.

Dependency chains کلیدی

- Docker run -> CLI -> Engine API -> dockerd/Moby -> containerd -> shim -> runc -> OCI config -> Linux syscalls/namespaces/cgroups/mounts
- Image build -> Dockerfile frontend -> LLB -> BuildKit solver -> worker/executor/snapshotter -> OCI image/index -> registry
- Image pull -> OCI Distribution -> descriptor graph -> content store -> unpack -> snapshots -> active rootfs -> runtime bundle
- Bridge network -> net namespace -> veth -> Linux bridge -> route/IP forwarding -> netfilter NAT/conntrack -> host NIC
- Resource limit -> Docker run config -> OCI Linux resources -> runc/libcontainer -> cgroup v2 controller files -> kernel scheduler/memory/IO
- Rootless Docker -> user namespace -> subuid/subgid -> rootless networking/storage/cgroup delegation -> daemon + containers without host root
- Compose -> Compose model/spec -> effective project config -> Docker Engine API objects -> networks/volumes/containers
- Swarm service -> desired state -> manager/Raft -> scheduler/task -> overlay/service discovery/routing mesh -> worker containers

فهرست فازها و ماژول‌ها

P0 - قرارداد پروژه، روش مطالعه و آزمایشگاه

- **META.DKR.01** - قرارداد تسلط Docker و Scope کل برنامه (Meta)
- **META.DKR.02** - محیط آزمایش و ابزارهای مشاهده سیستم (Foundation)
- **META.DKR.03** - سیستم Coverage و Snapshot کامل (Meta) Reference

Docker - P1 به عنوان محصول و runtime روزمره

- **DKR.01** - تاریخچه و مدل ذهنی (Core) Container
- **DKR.02** - نصب، Editions، Platforms و (Core) Distribution
- **DKR.03** - معماری (Core) Docker Engine: CLI -> API -> dockerd -> runtime stack
- **DKR.04** - Docker Object Model و (Core) Inspectability
- **DKR.05** - Container Lifecycle کامل (Core)
- **DKR.06** - I/O (Deep Core) و Process Model، PID 1، Signals، TTY
- **DKR.07** - Accelerators (Core+Internals) و Resource Constraints، OOM، PIDs، Devices
- **DKR.08** - Container Mutation Operations (Core) و Filesystem view

Distribution و P2 - Images، Dockerfile، BuildKit

- **DKR.09** - Image Mental Model: layers، config، manifest، digest (Core)
- **DKR.10** - Image Lifecycle (Core) و Naming، Tags، Digests، Content Addressability
- **DKR.11** - Dockerfile: Semantics کامل دستورات (Core)
- **DKR.12** - Context Types (Core) و Build Context، .dockerignore
- **DKR.13** - Reproducible Builds (Core) و Layering، Cache
- **DKR.14** - LLB (Deep Core) و BuildKit Architecture
- **DKR.15** - Bake (Core+Advanced) و Buildx، Builder Instances، Drivers
- **DKR.16** - Cross Building (Advanced) و Multi-platform Images
- **DKR.17** - Entitlements (Advanced) و Advanced Build Mounts، Secrets، SSH، Devices
- **DKR.18** - Provenance Attestations (Advanced) و Exporters، Cache Backends، SBOM
- **DKR.19** - Authentication (Core+Spec) و Registries، OCI Distribution، Docker Hub

Linux boundary تا Docker و P3 - Storage و Networking

- **DKR.20** - Persistence Model: Volumes، Bind Mounts، tmpfs (Core)
- **DKR.21** - Overlay Concepts (Deep Core) و Storage Drivers، Copy-on-Write
- **DKR.22** - Docker Engine (Advanced) در Snapshotters و containerd Image Store، Content Store
- **DKR.23** - Network Drivers (Core) و Docker Networking Model
- **DKR.24** - Bridge Networking Deep Dive: veth -> bridge -> routing -> NAT (Internals)
- **DKR.25** - Advanced Network Drivers: host/none/macvlan/ipvlan/IPv6 (Advanced)
- **DKR.26** - Swarm Data Plane (Advanced) و Overlay Networking
- **DKR.27** - Networking Troubleshooting Playbook (Production)

Swarm و P4 - Compose، Engine Administration، Observability، Security

- **DKR.28** - Specification (Core) و Compose Application Model

- Reuse (Advanced) و Compose Advanced: Merge, Include, Profiles, Watch - **DKR.29** •
- Host Integration (Production) و dockerd Administration - **DKR.30** •
- Upgrades (Production) و Versioning, API Compatibility, Deprecations - **DKR.31** •
- Diagnostics (Production) و Logs, Events, Metrics - **DKR.32** •
- Container/Daemon Security Model (Deep Core) - **DKR.33** •
- Rootless Docker (Deep Core) و User Namespaces, userns-remap - **DKR.34** •
- Software Supply Chain Security (Security) و Image - **DKR.35** •
- Trust Boundaries (Security) و Credentials, Secrets - **DKR.36** •
- PKI (Advanced) و Swarm Architecture: Nodes, Services, Tasks, Raft - **DKR.37** •
- DR (Advanced) و Swarm Operations: Overlay, Routing Mesh, Updates, Secrets - **DKR.38** •
- Extension APIs (Advanced+Contributor) و Docker Engine Plugin System - **DKR.39** •

Container زیر P5 - Linux internals

- file descriptors (Internals) و Linux Process Model, syscalls, /proc - **LNX.01** •
- API (Internals) و همه انواع: Linux Namespaces - **LNX.02** •
- ID Mapping Internals (Internals) و User Namespace - **LNX.03** •
- delegation (Internals) و cgroup v2: hierarchy, controllers - **LNX.04** •
- pivot_root (Internals) و Mounts, VFS, rootfs, chroot - **LNX.05** •
- OverlayFS Internals (Internals) - **LNX.06** •
- Containers (Foundation->Internals) برای Linux Networking Fundamentals - **LNX.07** •
- nftables/iptables (Internals) و veth, Linux Bridge, NAT, conntrack - **LNX.08** •
- devices (Internals) و Linux Security: capabilities, seccomp, LSM - **LNX.09** •
- Container Host Integration (Internals+Ops) و systemd, cgroups - **LNX.10** •

runc و P6 - OCI, containerd

- OCI Image Specification (Spec) - **OCI.01** •
- OCI Runtime Specification (Spec) - **OCI.02** •
- OCI Distribution Specification (Spec) - **OCI.03** •
- Non-image Content (Spec+Ecosystem) و OCI Artifacts, Referrers - **OCI.04** •
- Plugin/Service Model (Internals) و containerd Architecture - **CTR.01** •
- GC (Internals) و containerd Content Flow, Images, Snapshots, Leases - **CTR.02** •
- Shim (Internals) و containerd Containers vs Tasks, Runtime v2 - **CTR.03** •
- Runtime Lifecycle (Internals) و runc CLI, OCI Bundle - **RNC.01** •
- CRIU (Implementation) و libcontainer Internals, Bootstrap/Init - **RNC.02** •

P7 - Source Code Reading و ساخت اجزای مشابه

- SRC.MOB.01 - Build و Moby Repository Architecture (Contributor) و Test
- SRC.MOB.02 - Trace کردن Code Paths واقعی در Moby (Contributor)
- SRC.CLI.01 - Docker CLI Source و API Mapping (Contributor)
- SRC.BLD.01 - Solver، Worker، Cache، Executor و BuildKit Source (Contributor) و Frontend
- SRC.CMP.01 - Compose Specification و Compose Implementation Source (Contributor/Ecosystem)
- LAB.01 - ساخت (Capstone) Mini Container Runtime
- LAB.02 - ساخت (Capstone) Mini OCI Image/Registry Explorer
- LAB.03 - بازسازی Docker Bridge Networking بدون Docker (Capstone)

P8 - Cross-platform، Docker Product Ecosystem و قابلیت‌های جدید

- PLT.01 - Docker Desktop Architecture و Host Integration (Product)
- PLT.02 - Windows: WSL2، Hyper-V و Windows Containers (Product)
- PLT.03 - Docker Hub، Scout و Hardened Images به‌عنوان (Product) Product Surface
- PLT.04 - Model Runner، Sandboxes، MCP Toolkit (Awareness) و Emerging Docker AI/Agent Surface

P9 - Production، Failure Engineering، Ecosystem، Contribution و Interview

- OPS.01 - Production Engineering و Lifecycle Management (Production)
- OPS.02 - Performance Engineering (Production)
- OPS.03 - Failure Engineering و Troubleshooting Matrix (Production)
- OPS.04 - Ecosystem Comparison و Boundary Reasoning (Advanced)
- CONTRIB.01 - Contribution Workflow: از Issue تا Merged PR (Contributor)
- INT.01 - Interview Mastery: 30 seconds / 3 minutes / 30 minutes (Interview)
- INT.02 - Adversarial Interview و Production Scenario Drills (Interview)
- CAP.01 - Final Docker Mastery Capstone و Re-certification (Capstone)

P0 - قرارداد پروژه، روش مطالعه و آزمایشگاه

این فاز مشخص می‌کند «تسلط» در این پروژه دقیقاً چه معنی دارد، چگونه version drift کنترل می‌شود و محیطی می‌سازیم که بتوان عمداً آن را خراب کرد.

META.DKR.01 - قرارداد تسلط Docker و Scope کل برنامه

Meta :Depth

None / baseline :Prerequisites

پوشش اجباری

- تعریف شش سطح: <- User -> Power User -> Production Operator -> Troubleshooter -> Internals Expert -> Contributor.
- تفکیک «دانستن operation» از «حفظ syntax»؛ هدف: شناخت تمام capability ها و توانایی پیدا کردن reference دقیق.
- تعریف Core / Deep Internals / Product Ecosystem / Historical-Legacy tracks تا breadth باعث قربانی شدن depth نشود.
- اصل spiral learning و just-in-time prerequisites و Linux و networking هنگام نیاز باز می‌شوند و بعد عمیق‌تر بازگشت می‌کنیم.
- تعریف خروجی هر module: Study PDF، Scope Contract، lab، failure scenarios، source/spec reading و mastery evidence.

canonical منابع

- <https://docs.docker.com/> - Docker Documentation [DOCS]
- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://www.oreilly.com/library/view/docker-up/9781098131814/> - Docker: Up & Running, 3rd ed. - Sean P. Kane, Karl Matthias [BOOK_UR]
- <https://www.packtpub.com/en-us/product/docker-deep-dive-9781837028344> - Docker Deep Dive, 4th ed. - Nigel Poulton [BOOK_DD]

Evidence خاص این module

- نوشتن تعریف شخصی از «Docker expert» و معیارهای رد/قبول هر سطح.

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

META.DKR.02 - محیط آزمایش و ابزارهای مشاهده سیستم

Foundation: **Depth**

Prerequisites: **None / baseline**

پوشش اجباری

- حداقل یک Linux VM disposable، ترجیحاً دو VM برای شبکه و registry/Swarm؛ snapshot/restore برای بازگشت سریع.
- ابزارها: docker, dockerd, ctr, nerdctl(optional), runc, buildctl/buildx, ip, ss, nsenter, unshare, lsns, mount/findmnt, strace, lsof, tcpdump, iptables/nft, systemctl/journalctl, jq, curl
- ثبت baseline سخت‌افزار، Docker/BuildKit، /kernel، cgroup mode، filesystem، firewall backend، Docker/BuildKit، baseline سخت‌افزار، containerd/runc versions
- ساخت lab journal: برای هر آزمایش فرضیه، explanation، observation، command، cleanup ثبت شود.
- قاعده امنیتی: آزمایش‌های privileged/network/firewall فقط روی VM یا محیط disposable.

Labs / Experiments

- ایجاد snapshot اولیه VM و ثبت خروجی uname, docker info, docker version, mount, lsns و cgroup.controllers

منابع canonical

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/engine/release-notes/29/> - Docker Engine v29 release notes [RELEASE29]
- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS]
- <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Reference کامل Snapshot و Coverage سیستم - META.DKR.03

Meta :Depth

META.DKR.02 :Prerequisites

پوشش اجباری

- در هر baseline نسخه، خروجی docker help، docker buildx --help، docker compose --help، docker snapshot و --help dockerd شود.
- CLI commands بر اساس object و operation دسته‌بندی شوند: lifecycle, create/read/update/delete, transfer, inspect, debug, orchestration, build, security.
- Engine API endpoints و API version history به عنوان inventory مستقل ثبت شود؛ لازم نیست همه endpoint حفظ شوند اما هیچ capability ناشناخته نماند.
- Reference audit باید deprecated/experimental/hidden-behind-feature-flag capabilities را جدا علامت بزند.
- در هر ارتقای coverage matrix، major/minor، docs و release notes دوباره diff شود.

Labs / Experiments

- تولید inventory machine-readable command از help output و مقایسه آن با CLI reference.

منابع canonical

- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://docs.docker.com/reference/cli/dockerd/> - dockerd CLI reference [DOCKERD]
- <https://docs.docker.com/reference/api/engine/> - Docker Engine API reference [API]
- <https://docs.docker.com/engine/release-notes/29/> - Docker Engine v29 release notes [RELEASE29]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P0 Gate

- Gate P0: محیط قابل خراب‌کردن آماده است؛ baseline نسخه و command/API inventory ثبت شده؛ Definition of Done پذیرفته شده است.

Docker - P1 به عنوان محصول و runtime روزمره

از مدل ذهنی container تا معماری object model، Engine، lifecycle؛ این فاز باید تو را از کاربر معمولی به operator قابل اتکا برساند.

DKR.01 - تاریخچه و مدل ذهنی Container

Core :Depth

Prerequisites :META.DKR.01

پوشش اجباری

- مسئله‌ای که containers حل می‌کنند: density، reproducibility، isolation، packaging و delivery؛ محدودیت‌های portability.
- مقایسه process/container/VM/system container؛ تفاوت virtualization در سطح OS با hypervisor.
- Container یک kernel primitive منفرد نیست؛ مجموعه‌ای از process + namespaces + cgroups + mounts + credentials + security policy است.
- فرق Docker company/product، Docker Engine، Moby، OCI، containerd، runc، Compose و Kubernetes.
- Linux containers در برابر Windows containers؛ چیزهایی که shared kernel بودن تضمین نمی‌کند.

Labs / Experiments

- مقایسه process tree و namespace‌های یک process معمولی با container.

Failure scenarios

- توضیح false assumption‌های «container = lightweight VM» و «image = zip file».

منابع canonical

- <https://docs.docker.com/> - Docker Documentation [DOCS]
- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME]
- <https://www.oreilly.com/library/view/docker-up/9781098131814/> - Docker: Up & Running, 3rd ed. - Sean P. Kane, Karl Matthias [BOOK_UR]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.02 - نصب، Editions، Platforms و Distribution

Core :Depth

DKR.01, META.DKR.02 :Prerequisites

پوشش اجباری

- Docker Engine packages و install methods؛ Docker Desktop در برابر Engine؛ Linux/Windows/macOS .operational differences
- client/daemon version و docker context، DOCKER_HOST، local Unix socket، TCP/SSH remote access .differences
- package-managed vs static binaries؛ rootful vs rootless deployment shape و implications برای .bundled components
- نسخه‌ها، compatibility، release channels و pinning؛ تشخیص Buildx، Compose، API، CLI، Engine، containerd و BuildKit، runc versions
- Baseline فعلی سند: Engine 29.x؛ اما هر lab باید version واقعی محیط را ثبت کند.

Labs / Experiments

- نصب روی Linux VM؛ ساخت یک context دوم برای daemon remote آزمایشی.

Failure scenarios

- client/daemon version mismatch
- docker.sock روی permission denied
- remote daemon نامن TCP.

canonical منابع

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/engine/release-notes/29/> - Docker Engine v29 release notes [RELEASE29]
- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://docs.docker.com/engine/security/rootless/> - Docker rootless mode [ROOTLESS]
- <https://docs.docker.com/desktop/> - Docker Desktop manual [DESKTOP]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Docker Engine: CLI -> API -> dockerd -> runtime stack معماری - DKR.03

Core :Depth

DKR.01, DKR.02 :Prerequisites

پوشش اجباری

- Docker objects و Client-server architecture، dockerd responsibilities
- Docker ؛Unix socket/TCP، REST Engine API، version negotiation، request/response/error model
- endpoint/TLS selection precedence و contexts، DOCKER_HOST/DOCKER_CONTEXT
- جایگاه shim، containerd، Moby و runc در path اجرای Linux container؛ تمایز logical architecture و packaging؛ و topologiesهای version-sensitive مانند embedded-containerd experimental را بدون قاطعی کردن logical contract با deployment topology تحلیل کن.
- Events و state transitions؛ daemon responsibilities در network/volume/image/runtime orchestration.
- مرزهایی که Docker به kernel delegation می‌دهد و چیزهایی که خودش implement می‌کند.

Labs / Experiments

- با curl روی /var/run/docker.sock version/info/containers endpoints/ یک container را بدون docker CLI create/start/delete کن.

Failure scenarios

- API version mismatch
- daemon unavailable
- stale state after daemon restart

canonical منابع

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/reference/api/engine/> - Docker Engine API reference [API]
- <https://github.com/containerd/> - containerd runtime v2 architecture [CONTAINERD_RUNTIME]
[containerd/blob/main/docs/runtime-v2.md](https://github.com/containerd/blob/main/docs/runtime-v2.md)
- <https://github.com/moby/moby> - Moby repository [MOBY]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Inspectability و DKR.04 – Docker Object Model

Core :Depth

DKR.03 :Prerequisites

پوشش اجباری

- Objects: containers, images, networks, volumes, plugins, contexts, services, nodes, secrets, configs
- builders: lifecycle awareness: object-specific prune/system-df/system-prune
- Names/IDs/digests/labels/annotations و lifecycle متفاوت هر object.
- object metadata و docker inspect. Go templates, filtering, formatting, labels
- فرق runtime state و desired configuration: derived status و operations مکمل مانند /diff/cp/export
- object mutation/transfer دقیق boundary و import/save/load/commit/top/stats/wait/update
- قاعده operational: قبل از mutation state را inspect و evidence جمع کن.

Labs / Experiments

- برای container/image/network/volume خروجی inspect را map و فیلدهای مهم را مستند کن.

canonical منابع

- <https://docs.docker.com/reference/cli/docker/> – Docker CLI reference [CLI]
- <https://docs.docker.com/reference/api/engine/> – Docker Engine API reference [API]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.05 - Container Lifecycle کامل

Core :Depth

DKR.04 :Prerequisites

پوشش اجباری

- create, start, run, exec, attach, detach, pause/unpause, stop, kill, restart, wait, rename, remove
- export/import/commit/cp و تفاوتشان با image build/registry workflows؛ موارد مناسب و anti-pattern ها.
- ordering و AutoRemove، restart policy، container state machine، exit status، health status
- فرق stop timeout و kill signal؛ STOP SIGNAL و custom stop signal
- تفاوت docker exec با شروع process اولیه و اثر namespace/cgroup membership

Labs / Experiments

- lifecycle یک container را با events و inspect همزمان trace کن.

Failure scenarios

- container stuck stopping
- exec fails when init exited
- restart loop
- paused container behavior

canonical منابع

<https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]

<https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

I/O و DKR.06 - Process Model, PID 1, Signals, TTY

Deep Core :Depth

DKR.05 :Prerequisites

پوشش اجباری

- PID 1 semantics در namespace، signal handling، zombie reaping و init helper.
- ENTRYPOINT/CMD در runtime؛ exec form vs shell form و اثر shell بر signal propagation.
- detach keys و process groups/sessions، stdin/stdout/stderr، pseudo-TTY، attach vs logs.
- graceful shutdown و application contracts؛ stop signal/timeouts و orchestrator expectations.
- چرا daemonizing داخل container معمولاً anti-pattern است؛ one-process guideline و استثناها.

Labs / Experiments

- برنامه کوچک بساز که SIGTERM را log کند؛ تفاوت shell-form و exec-form را اندازه بگیر.

Failure scenarios

- SIGTERM به app نمی‌رسد
- zombie accumulation
- interactive container hang

canonical منابع

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/reference/dockerfile/> - Dockerfile reference [DOCKERFILE]
- <https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source path code نیز trace شود.

Accelerators و DKR.07 – Resource Constraints, OOM, PIDs, Devices

Core+Internals :Depth

DKR.05 :Prerequisites

پوشش اجباری

- failure و Memory hard/soft limits, swap semantics, OOM killer, oom score, memory pressure interpretation.
- .limit/weight/reservation و تفاوت CPU shares/weight, quota/period, cpuset, realtime concepts.
- .device access و PIDs limits, ulimits, blkio/IO controls.
- .device injection با relationship و GPU access, NVIDIA Container Toolkit awareness.
- .security implications و build-time CDI awareness ؛declarative device exposure برای CDI concept.

Labs / Experiments

- memory limit را container و OOM ایجاد کن؛ CPU throttling را مشاهده کن؛ pids limit بشکن.

Failure scenarios

- host OOM vs container OOM.
- swap misunderstanding.
- .resource flag unsupported by host kernel.

منابع canonical

- https://docs.docker.com/engine/containers/resource_constraints/ - Docker container resource constraints [RESOURCE]
- <https://docs.docker.com/engine/containers/gpu/> - Docker Engine GPU access [GPU]
- <https://docs.docker.com/build/building/cdi/> - Docker Build CDI [CDI]
- <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Container Mutation Operations و DKR.08 - Filesystem view

Core :Depth

DKR.05 :Prerequisites

پوشش اجباری

- mutation visibility و concept، docker diff در سطح container writable layer، copy-on-write
- security edge cases و docker cp semantics، ownership/path/symlink considerations
- export/import در برابر save/load در برابر commit؛ چه metadataهایی حفظ یا از دست می‌روند.
- ephemeral state و read-only root filesystem، tmpfs paths
- Operational rule: runtime mutation را با immutable build pipeline مقایسه و anti-patternها را تشخیص بده.

Labs / Experiments

- یک image را mutate کن؛ diff/export/commit را مقایسه و metadata تفاوت را ثبت کن.

canonical منابع

- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://docs.docker.com/engine/storage/> - Docker Engine storage [STORAGE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P1 Gate

- Gate P1: باید بتوانی یک container را بدون CLI از API بسازی، lifecycle و signal path را توضیح بدهی، resource failureها را diagnose کنی و object state را دقیق inspect کنی.

Distribution و P2 – Images, Dockerfile, BuildKit

از image به عنوان artifact روزمره تا attestations، multi-platform، cache، LLB، OCI graph و registry protocol.

DKR.09 – Image Mental Model: layers, config, manifest, digest

Core :Depth

DKR.04 :Prerequisites

پوشش اجباری

- immutable content identity در برابر mutable naming و image vs container، repository/tag/digest
- platform metadata و layer changesets، image config، history، rootfs ordering
- multi-platform selection :manifest vs image index/manifest list
- compressed blob digest، uncompressed DiffID و chain relationships در سطح مفهومی.
- docker image inspect/history/save/load و mapping به OCI concepts.

Labs / Experiments

- docker save یک image را باز کن و manifest/config/layers را با jq/tar بررسی کن.

Failure scenarios

- tag drift
- wrong platform image
- digest mismatch mental model

منابع canonical

- <https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE]
- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m :teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Image Lifecycle و DKR.10 - Naming, Tags, Digests, Content Addressability

Core :Depth

DKR.09 :Prerequisites

پوشش اجباری

- image reference grammar: registry/namespace/repository:tag@digest
- garbage collection/prune و pull policies, local cache, dangling/unused images
- trust/reproducibility implications و content-addressable identity, deduplication
- registry در tag immutability policies; pinning by digest و platform-aware pulls
- image import/export/save/load/pull/push/remove/prune semantics

Labs / Experiments

- همان image را با tag و digest pull کن و identity را مقایسه کن.

canonical منابع

- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE]
- <https://specs.opencontainers.org/distribution-spec/> - OCI Distribution Specification [OCI_DIST]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.11 - Dockerfile: Semantics کامل دستورات

Core :Depth

DKR.09 :Prerequisites

پوشش اجباری

- FROM, ARG, ENV, RUN, COPY, ADD, WORKDIR, USER, EXPOSE, VOLUME, LABEL, SHELL, STOPSIGNAL, HEALTHCHECK, ONBUILD, CMD, ENTRYPOINT
- scope و variable replacement; shell vs exec forms; syntax directive و parser directives
- COPY/ADD source semantics, ownership/permissions, --link/advanced features, version
- EXPOSE documentation vs port publishing; UID/GID portability و USER

• `docker run/Compose` در `override behavior` و `CMD/ENTRYPOINT composition`.

Labs / Experiments

• برای هر `instruction` حداقل یک `Dockerfile` `minimal` و `inspect/history evidence` بساز.

Failure scenarios

- ARG/ENV scope surprise
- COPY path/permission errors
- CMD/ENTRYPOINT override confusion

canonical منابع

• <https://docs.docker.com/reference/dockerfile/> - Dockerfile reference [DOCKERFILE]

Definition of Done مشترک

• توضیح بدون `note + diagram` از حافظه؛ اجرای `lab`؛ ایجاد حداقل یک `failure`؛ توضیح ارتباط با `modules` مجاور؛ پاسخ `30s/3m/30m` `teach-back` به `study partner`. برای `Internals/Contributor`: حداقل یک `spec/source` `code path` نیز `trace` شود.

Context Types و DKR.12 - Build Context، .dockerignore

Core :Depth

DKR.11 :Prerequisites

پوشش اجباری

- `security boundary` و `context transfer`؛ `local directory/tar/stdin/Git/remote contexts`.
- `Dockerfile-specific ignore files` و `dockerignore matching semantics`.
- `cross-context COPY/mount patterns` و `named contexts`.
- چرا `secrets` نباید داخل `context` بیایند؛ `cache retention` و `accidental leakage`.
- `reproducibility implications` و `context size/performance`.

Labs / Experiments

• یک `secret` را عمداً وارد `context` کن و سپس با `dockerignore` و `secret mount` اصلاح کن.

Failure scenarios

- unexpected cache invalidation
- huge context transfer

• secret leaked into layer

منابع canonical

• <https://docs.docker.com/build/> - Docker Build manual [BUILD]

• <https://docs.docker.com/reference/dockerfile/> - Dockerfile reference [DOCKERFILE]

Definition of Done مشترک

• توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source؛ code path نیز trace شود.

Reproducible Builds و DKR.13 - Layering, Cache

Core :Depth

DKR.11, DKR.12 :Prerequisites

پوشش اجباری

- package-manager patterns و cache keys، invalidation، instruction ordering
- multi-stage builds، target stages، scratch/distroless/minimal bases
- determinism/reproducibility: pin dependencies، timestamps، external downloads، digest pinning
- inline/registry/local/GHA awareness؛ external cache strategies و cache mounts
- image size vs attack surface vs debuggability trade-offs

Labs / Experiments

• یک build کند را profile و با cache/multi-stage بهینه کن؛ reproducibility را با دو build مقایسه کن.

Failure scenarios

- stale package cache
- non-reproducible download
- cache poisoning assumptions

منابع canonical

• <https://docs.docker.com/build/> - Docker Build manual [BUILD]

• <https://docs.docker.com/build/buildkit/> - Docker BuildKit manual [BUILDKIT]

• <https://docs.docker.com/reference/dockerfile/> - Dockerfile reference [DOCKERFILE]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source؛ code path نیز trace شود.

LLB و DKR.14 – BuildKit Architecture

Deep Core :Depth

DKR.13 :Prerequisites

پوشش اجباری

- /BuildKit daemon، frontends، solver، workers، executors، snapshotters، cache manager، exporters، importers.
- LLB به عنوان content-addressable dependency graph؛ operation DAG و parallelism.
- BuildKit sessions، sources؛ Dockerfile frontend -> LLB -> solver -> worker -> result/export pipeline، workers/executors/snapshotters و GC policy boundaries.
- build execution در network/security modes و entitlements، sandboxing.
- تفاوت legacy builder architecture با BuildKit و دلیل performance/extensibility improvements.

Labs / Experiments

- LLB یک build را dump/inspect کن؛ independent stages را طوری بساز که parallel execution قابل مشاهده باشد.

Failure scenarios

- builder unavailable
- frontend version incompatibility
- cache miss unexplained

canonical منابع

- <https://docs.docker.com/build/buildkit/> - Docker BuildKit manual [BUILDKIT]
- <https://github.com/moby/buildkit> - BuildKit repository [BUILDKIT_REPO]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source؛ code path نیز trace شود.

Bake و DKR.15 – Buildx, Builder Instances, Drivers

Core+Advanced :Depth

DKR.14 :Prerequisites

پوشش اجباری

- builder instances/nodes/platforms ؛BuildKit با relation و Buildx CLI plugin
- .capability differences ؛drivers: docker, docker-container, remote, kubernetes awareness
- .resource configuration و builder creation/inspection/bootstrap/use/stop/remove
- .CI reuse و target groups, matrices/variables ؛Bake/HCL/Compose-derived build definitions
- .cache/exporter behavior differences based on driver and image store

Labs / Experiments

- docker-container builder مستقل بساز، inspect کن و با default docker driver مقایسه کن.

canonical منابع

- <https://docs.docker.com/build/> - Docker Build manual [BUILD]
- <https://docs.docker.com/build/buildkit/> - Docker BuildKit manual [BUILDKIT]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Cross Building و DKR.16 – Multi-platform Images

Advanced :Depth

DKR.09, DKR.15 :Prerequisites

پوشش اجباری

- .manifest selection و image index ؛platform tuple: OS/architecture/variant
- .cross-compilation strategies و QEMU/binfmt emulation, native multi-node builders
- .platform-aware Dockerfiles و BUILDPLATFORM/TARGETPLATFORM args
- .architecture-specific dependencies و performance/correctness trade-offs
- .push/load output choices و local image store limitations/behavior

Labs / Experiments

• amd64+arm64 image بساز و index/manifests را inspect کن.

Failure scenarios

- exec format error
- wrong platform selected
- .emulation-only failure

canonical منابع

- <https://docs.docker.com/build/> - Docker Build manual [BUILD]
- <https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE]
- <https://docs.docker.com/engine/storage/containerd/> - containerd image store with Docker Engine [CONTAINERD_STORE]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Entitlements و DKR.17 - Advanced Build Mounts, Secrets, SSH, Devices

Advanced :Depth

DKR.14 :Prerequisites

پوشش اجباری

- RUN --mount types: cache, secret, ssh, bind, tmpfs و lifetime/security semantics.
- secret values در layer/history/cache نباید persist شوند؛ verification techniques.
- SSH forwarding برای private source access بدون key baking.
- insecure/security entitlements و build network modes و threat model.
- CDI devices/GPU during build و labs-only/feature-gated syntax awareness.

Labs / Experiments

• private dependency pattern را با secret/ssh mount بساز و ثابت کن secret در image نیست.

Failure scenarios

- secret persisted accidentally

- SSH agent unavailable
- insecure entitlement unintentionally enabled

canonical منابع

- <https://docs.docker.com/build/> - Docker Build manual [BUILD]
- <https://docs.docker.com/build/building/cdi/> - Docker Build CDI [CDI]
- <https://docs.docker.com/build/buildkit/> - Docker BuildKit manual [BUILDKIT]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Provenance Attestations و DKR.18 - Exporters, Cache Backends, SBOM

Advanced :Depth

DKR.14, DKR.16 :Prerequisites

پوشش اجباری

- exporters: image/registry/local/tar/OCI/docker و تفاوت artifact outputs.
- cache import/export backends و cache portability.
- provenance و SBOM attestations؛ in-toto/SLSA awareness و attachment to image index.
- CI policy consumption و provenance modes، reproducibility metadata.
- relation با registry support و Scout policies، containerd image store.

Labs / Experiments

- image را با SBOM/provenance بساز، push و attestations را inspect/extract کن.

Failure scenarios

- attestation lost on incompatible output
- no base-image provenance data
- registry feature mismatch

canonical منابع

- <https://docs.docker.com/build/metadata/attestations/> - Docker build attestations [BUILD_ATTEST]
- <https://docs.docker.com/scout/> - Docker Scout manual [SCOUT]

<https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE] •

Definition of Done مشترک

• توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Authentication و DKR.19 - Registries, OCI Distribution, Docker Hub

Core+Spec :Depth

DKR.09, DKR.10 :Prerequisites

پوشش اجباری

• Registry HTTP API v2 concepts و registry/repository/blob/manifest/index vocabulary
• pull/push workflow، HEAD/GET/POST/PATCH/PUT، resumable blobs، deduplication و error codes
• auth challenges/tokens، docker login/logout، credential stores/helpers، PATs، device-code/browser
• flows در صورت registry mirrors/insecure registry trust و platform relevance، least privilege
• Docker Hub repositories/tags/access/webhooks/trusted content/API؛ awareness؛ rate/usage limits
• OCI artifacts/referrers و non-image artifacts؛ registry interoperability

Labs / Experiments

• local registry راه بینداز؛ manifest و blob را با HTTP مستقیم بگیر؛ یک blob digest را verify کن.

Failure scenarios

• challenge 401
• manifest unknown
• blob upload interrupted
• rate limit

منابع canonical

<https://specs.opencontainers.org/distribution-spec/> - OCI Distribution Specification [OCI_DIST] •

<https://docs.docker.com/docker-hub/> - Docker Hub manual [HUB] •

<https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI] •

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P2 Gate

• Gate P2: باید یک OCI image را دستی تشریح کنی، Dockerfile semantics را دفاع کنی، Dockerfile->LLB flow را توضیح بدهی، multi-platform image و attestations بسازی و registry را بدون docker pull با HTTP بررسی کنی.

Linux boundary تا Docker از Networking و P3 – Storage

این فاز دو حوزه‌ای را عمیق می‌کند که بیشترین production incident را می‌سازند: packet path و persistence.

DKR.20 – Persistence Model: Volumes, Bind Mounts, tmpfs

Core :Depth

DKR.08 :Prerequisites

پوشش اجباری

- فرق tmpfs, bind mounts, named/anonymous volumes, writable layer و storage plugins.
- --mount vs -v semantics: ownership, SELinux labeling, readonly, subpaths و propagation awareness.
- volume lifecycle مستقل از container: backup/restore/migration patterns.
- security risks و host coupling/ security risks و path portability.
- behavior memory-backed و tmpfs و secrets/ephemeral data patterns.

Labs / Experiments

- برای یک DB volume backup/restore انجام بده؛ bind/volume/tmpfs behavior را مقایسه کن.

Failure scenarios

- permission mismatch UID/GID
- mount obscures existing files
- volume deleted/misidentified

منابع canonical

- <https://docs.docker.com/engine/storage/> – Docker Engine storage [STORAGE]
- <https://docs.docker.com/reference/cli/docker/> – Docker CLI reference [CLI]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Overlay Concepts و DKR.21 - Storage Drivers، Copy-on-Write

Deep Core :Depth

DKR.09, DKR.20 :Prerequisites

پوشش اجباری

- .overlay2، vfs، btrfs/zfs awareness و classic graph/storage-driver model
- .performance implications و copy-on-write، lower/upper/merged/work concepts
- .copy-up و write amplification :container writable layer vs persistent data
- .migration between storage backends و backing filesystem requirements
- .disk accounting pitfalls و docker system df/prune

Labs / Experiments

- .observe کن، copy-up و file modification را روی overlay2 directory layout inspect کن؛

Failure scenarios

- disk full
- inode exhaustion
- .storage driver switch hides old data

canonical منابع

- <https://docs.docker.com/engine/storage/> - Docker Engine storage [STORAGE]
- <https://docs.kernel.org/filesystems/overlayfs.html> - Linux kernel - OverlayFS [LNX_OVERLAY]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Docker در Snapshotters و DKR.22 - containerd Image Store, Content Store Engine

Advanced :Depth

DKR.21, DKR.14 :Prerequisites

پوشش اجباری

- classic graph drivers با fresh Engine 29 در containerd image store.
- why disk usage differs :unpacked snapshots در برابر compressed content store.
- advanced lazy/p2p snapshotter awareness :overlayfs default و snapshotter plugin model.
- migration/visibility behavior و data roots :attestations support و multi-platform images.
- relation Docker image metadata <-> containerd content/images/snapshots.

Labs / Experiments

- فعال بودن containerd image store را verify کن؛ content/snapshot state را با ctr و docker مقایسه کن.

Failure scenarios

- backend switch makes objects invisible
- containerd root partition fills
- snapshotter failure

منابع canonical

- <https://docs.docker.com/engine/storage/containerd/> - containerd image store with Docker Engine [CONTAINERD_STORE]
- <https://github.com/containerd/containerd/blob/main/docs/content-flow.md> - containerd content flow [CONTAINERD_FLOW]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor حداقل یک spec/source code path نیز trace شود.

Network Drivers و DKR.23 – Docker Networking Model

Core :Depth

DKR.04 :Prerequisites

پوشش اجباری

- Docker implementation: network, endpoint, sandbox; IPAM concepts در CNM-style object concepts
- .name resolution و embedded DNS ؛default bridge vs user-defined bridge
- -internal/attachable/IPv4/IPv6/IPAM/default ؛drivers: bridge, host, none, overlay, macvlan, ipvlan
- .plugin-driver boundary و address-pools/MTU options
- .multi-network attachment و container interfaces/routes/gateway
- EXPOSE در برابر publish ؛userland-proxy, ephemeral host ports, host IP binding, port mapping syntax
- .host firewall با interaction و awareness, port-allocation conflicts

Labs / Experiments

- چند network بساز و route/DNS/interface changes را داخل container inspect کن.

Failure scenarios

- wrong network attachment
- name resolution unexpected
- .published port bound to wrong interface

canonical منابع

- <https://docs.docker.com/engine/network/> – Docker Engine networking [NETWORK]
- <https://docs.docker.com/reference/cli/docker/> – Docker CLI reference [CLI]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m ؛teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.24 – Bridge Networking Deep Dive: veth -> bridge -> routing -> NAT

Internals :Depth

DKR.23 :Prerequisites

پوشش اجباری

- default gateway path و network namespace، veth pair، Linux bridge، L2/L3 boundaries
- port publishing packet path و IP forwarding، NAT/masquerade، DNAT/SNAT
- DOCKER chains/history awareness :Docker-created rules و iptables/nftables backend
- conntrack role و hairpin/local host-to-container/container-to-internet flows
- user-defined bridge در DNS forwarding/internal resolver behavior

Labs / Experiments

- packet path یک curl از host به published port و از container به Internet را با tcpdump/rules trace کن.

Failure scenarios

- IP forwarding disabled
- NAT rule missing
- host firewall conflict
- hairpin edge case

منابع canonical

- <https://docs.docker.com/engine/network/> - Docker Engine networking [NETWORK]
- <https://docs.docker.com/engine/network/packet-filtering-firewalls/> - Docker packet filtering and firewalls [FIREWALL]
- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNK_NS]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.25 – Advanced Network Drivers: host/none/macvlan/ipvlan/IPv6

Advanced :Depth

DKR.23, DKR.24 :Prerequisites

پوشش اجباری

- manual plumbing و none network ؛host networking isolation trade-off
- host communication caveat و macvlan modes, parent interface, L2 identity
- routing implications و ipvlan L2/L3 modes
- dual-stack considerations و IPv6 addressing, forwarding, NAT differences
- route selection و internal networks, multi-homing

Labs / Experiments

- macvlan یا ipvlan در VM lab بساز؛ none container را دستی network کن.

Failure scenarios

- host cannot reach macvlan child
- upstream switch limitations
- IPv6 forwarding/DNS issues

canonical منابع

- <https://docs.docker.com/engine/network/> - Docker Engine networking [NETWORK]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Swarm Data Plane و DKR.26 – Overlay Networking

Advanced :Depth

DKR.23 :Prerequisites

پوشش اجباری

- VXLAN concept ؛multi-host connectivity برای overlay network purpose
- data-plane packet encapsulation در برابر control-plane membership/service discovery

- Swarm published ports و ingress/routing mesh concepts.
- standalone containers و attachable overlay :encryption option و overhead.
- troubleshooting boundaries و network MTU، underlay dependence.

Labs / Experiments

- دو node Swarm lab و overlay بساز؛ VXLAN traffic را capture کن.

Failure scenarios

- overlay packets blocked
- MTU fragmentation
- routing mesh confusion

canonical منابع

- <https://docs.docker.com/engine/network/> - Docker Engine networking [NETWORK]
- <https://docs.docker.com/engine/swarm/> - Docker Swarm mode [SWARM]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.27 - Networking Troubleshooting Playbook

Production :Depth

DKR.24, DKR.25 :Prerequisites

پوشش اجباری

- <- decision tree: process/listen socket -> namespace/interface -> route -> DNS -> firewall/NAT
- .remote network -> application protocol
- ابزارها: ss, ip addr/route/link/neigh, nsenter, tcpdump, dig/getent, curl/nc, conntrack, iptables-save/nft, .list ruleset
- .DNS failure classification و etc/resolv.conf behavior. host resolvers/ و container DNS
- .ephemeral ports awareness و MTU/MSS. asymmetric routing, conntrack exhaustion
- .why published ports may bypass expected host firewall policy و firewall/ufw integration pitfalls

Labs / Experiments

• پنج failure سناریو بساز: DNS, wrong bind address, missing route, firewall, MTU.

Failure scenarios

- DNS timeout
- connection refused vs timeout
- conntrack full
- ufw assumption

منابع canonical

<https://docs.docker.com/engine/network/packet-filtering-firewalls/> - Docker packet filtering and firewalls [FIREWALL]

<https://docs.docker.com/engine/network/> - Docker Engine networking [NETWORK]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P3 Gate

• Gate P3: باید bridge networking را بدون Docker بازسازی کنی، packet path را trace کنی، storage backend ها را توضیح بدهی و permission/disk/network incidents را به صورت systematic debug کنی.

P4 – Compose, Engine Administration, Observability Swarm و Security

از یک host و container منفرد به production security، daemon operations، application model و orchestration داخلی Docker.

Specification و DKR.28 – Compose Application Model

Core :Depth

DKR.05, DKR.20, DKR.23 :Prerequisites

پوشش اجباری

- project identity و Compose application model: services, networks, volumes, configs, secrets
- Compose Specification current model و legacy 2.x/3.x history :schema validation و canonical config
- build/image, command/entrypoint, environment/env_file, ports/expose, volumes, networks
- healthcheck, restart/deploy distinctions
- readiness vs startup ordering و depends_on semantics
- /Compose CLI lifecycle: up/down/create/start/stop/restart/exec/run/logs/ps/pull/push/build/config
- .events

Labs / Experiments

- یک سه‌سرویس app با healthcheck/network/volume بساز و docker compose config را canonicalize کن.

Failure scenarios

- depends_on mistaken for readiness
- env interpolation mismatch
- orphan resources

canonical منابع

- <https://docs.docker.com/reference/compose-file/> - Compose file reference / Compose Specification implementation [COMPOSE]
- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Reuse و DKR.29 – Compose Advanced: Merge, Include, Profiles, Watch

Advanced :Depth

DKR.28 :Prerequisites

پوشش اجباری

- multiple files و merge rules؛ overrides و environment-specific composition.
- include، anchors vs spec-level reuse، fragments/extensions، profiles، model normalization.
- environment env/env_file/process، variable interpolation precedence، debugging effective config.
- dry-run، develop/watch، lifecycle hooks/health/dependencies، scale، one-off jobs و /conversion.
- awareness integration provider-style/bridge؛ هر feature جدید Compose باید در META.DKR.03 current inventory شود.
- semantics configs/secrets و portability gap با orchestrators.

Labs / Experiments

- compose layers prod+dev+base و final config را diff کن؛ profile و watch استفاده کن.

Failure scenarios

- unexpected merge
- wrong env precedence
- .bind mount performance on Desktop

canonical منابع

- <https://docs.docker.com/reference/compose-file/> - Compose file reference / Compose Specification implementation [COMPOSE]
- <https://docs.docker.com/desktop/> - Docker Desktop manual [DESKTOP]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Host Integration و DKR.30 – dockerd Administration

Production :Depth

DKR.03, META.DKR.02 :Prerequisites

پوشش اجباری

- .conflict rules و daemon startup, systemd units/socket activation, daemon.json vs flags
- .remote daemon security و Unix/TCP sockets, TLS mutual auth, SSH contexts
- .proxies, DNS defaults, registry mirrors/insecure registries :containerd data location و data-root
- .live-restore, default runtimes, log defaults, iptables/nftables settings, shutdown/restart behavior
- .daemon-wide transfer/concurrency controls و stop-timeout defaults
- .config validation و multiple daemons/test daemons, debug mode, system logs

Labs / Experiments

- daemon config را تغییر و validate کن؛ TLS-protected remote daemon بساز.

Failure scenarios

- daemon won't start due config conflict
- data root full
- .remote socket exposed unauthenticated

منابع canonical

- <https://docs.docker.com/reference/cli/dockerd/> – dockerd CLI reference [DOCKERD]
- <https://docs.docker.com/engine/> – Docker Engine manual [ENGINE]
- <https://docs.docker.com/engine/storage/containerd/> – containerd image store with Docker Engine [CONTAINERD_STORE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Upgrades و DKR.31 - Versioning, API Compatibility, Deprecations

Production :Depth

DKR.30, META.DKR.03 :Prerequisites

پوشش اجباری

- API version negotiation/history و Engine/CLI/API version relationships
- behavior changes و release-note-driven migration ؛ deprecated/removed features
- why source و packages/static binaries در bundled component versions (BuildKit/containerd/runc)
- debugging needs exact versions
- upgrade/downgrade planning, rollback constraints, storage/network compatibility
- deprecation examples ؛ production policy و feature flags/experimental features
- release-specific verification با legacy links و cgroup v1

Labs / Experiments

- دو release note متوالی را diff و breaking/deprecation checklist تولید کن.

Failure scenarios

- deprecated feature breaks upgrade
- API client hardcodes newer endpoint

canonical منابع

- <https://docs.docker.com/engine/release-notes/29/> - Docker Engine v29 release notes [RELEASE29]
- <https://docs.docker.com/reference/api/engine/> - Docker Engine API reference [API]
- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Diagnostics و DKR.32 - Logs, Events, Metrics

Production :Depth

DKR.05, DKR.30 :Prerequisites

پوشش اجباری

- json-file/local/syslog/journald/fluentd/gelf/awslogs/splunk ؛ logging drivers و container stdout/stderr .awareness
- .loss/backpressure trade-offs و log rotation, blocking/non-blocking modes
- .lifecycle incidents و correlation و docker events event stream
- .daemon metrics/Prometheus exposure awareness ؛ cgroup metrics و docker stats
- .OS-level telemetry integration و daemon logs, inspect, system df/info

Labs / Experiments

- یک logging driver alternative آزمایش کن؛ event stream و stats را با incident timeline correlate کن.

Failure scenarios

- disk filled by logs
- logging driver blocks app
- .missing logs after crash

منابع canonical

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

DKR.33 – Container/Daemon Security Model

Deep Core :Depth

DKR.06, DKR.07 :Prerequisites

پوشش اجباری

- Docker daemon/root authority و خطر docker.sock؛ access to daemon ~ host root در rootful model.
- privileged containers، device mounts، host namespaces، bind mounts و breakout risk amplification.
- Linux capabilities default set/drop/add؛ no-new-privileges.
- seccomp default profile و syscall filtering؛ AppArmor/SELinux LSM integration.
- kernel attack surface، masked/readonly paths، user selection، read-only FS و host patching؛ /archive.
- copy/extraction paths و historical security failures به عنوان reminder برای trust-boundary analysis.

Labs / Experiments

- capabilities را drop/add کن و behavior را مشاهده کن؛ seccomp-denied syscall demo بساز.

Failure scenarios

- container works only privileged
- socket mounted into untrusted container
- LSM policy denial

منابع canonical

- <https://man7.org/linux/man-pages/man7/capabilities.7.html> - Linux capabilities(7) [LNX_CAP]
- <https://man7.org/linux/man-pages/man2/seccomp.2.html> - Linux seccomp(2) [LNX_SECCOMP]
- <https://docs.docker.com/engine/security/rootless/> - Docker rootless mode [ROOTLESS]
- <https://docs.docker.com/engine/security/userns-remap/> - Docker user namespace remapping [USERNS]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Rootless Docker و DKR.34 - User Namespaces، userns-remap

Deep Core :Depth

DKR.33 :Prerequisites

پوشش اجباری

- ownership translation و user namespace UID/GID mappings، subordinate IDs
- storage/mount limitations ؛containers remapped و userns-remap: daemon rootful
- rootlesskit/network/storage implications ؛user namespace داخل containers و rootless: daemon
- cgroup delegation/systemd considerations ؛version-sensitive drivers و rootless networking evolution
- فرق rootless با "USER non-root" داخل container و با Desktop isolation

Labs / Experiments

- rootless daemon نصب کن؛ host/container UID mapping را با files و proc بررسی کن.

Failure scenarios

- subuid/subgid missing
- privileged port/device limitations
- bind mount ownership confusion

canonical منابع

- <https://docs.docker.com/engine/security/rootless/> - Docker rootless mode [ROOTLESS]
- <https://docs.docker.com/engine/security/userns-remap/> - Docker user namespace remapping [USERNS]
- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS]
- <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Software Supply Chain Security و DKR.35 - Image

Security :Depth

DKR.18, DKR.19, DKR.33 :Prerequisites

پوشش اجباری

- patch cadence و base-image trust, digest pinning, minimal/distroless images
- policy enforcement و SBOM, provenance, attestations, signatures/VEX concepts
- license checks و Docker Scout CVE analysis, policy evaluation, base-image freshness
- shared responsibility و Docker Hardened Images concepts, signed metadata, SLSA/VEX
- CI admission gates و secret scanning, dependency lifecycle

Labs / Experiments

- یک image آسیب‌پذیر scan کن؛ policy برای non-root/SBOM/provenance اجرا و remediate کن.

Failure scenarios

- "false confidence from "0 CVE
- stale base digest
- missing provenance

canonical منابع

- <https://docs.docker.com/scout/> - Docker Scout manual [SCOUT]
- <https://docs.docker.com/dhi/> - Docker Hardened Images [DHI]
- <https://docs.docker.com/build/metadata/attestations/> - Docker build attestations [BUILD_ATTEST]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Trust Boundaries و DKR.36 – Credentials, Secrets

Security :Depth

DKR.19, DKR.35 :Prerequisites

پوشش اجباری

- tokens vs passwords ؛credential helpers/stores و docker login credential storage
- Build secrets/SSH در برابر runtime secrets و environment-variable leakage
- Swarm secrets/configs encryption-at-rest model و access lifecycle
- TLS certs برای daemon/registry و trust store management
- CI credentials scope، short-lived tokens و avoiding docker.sock exposure

Labs / Experiments

- credential helper configure کن؛ secret را در env/history/logs leak و سپس اصلاح کن.

Failure scenarios

- plaintext credentials in config
- secret baked in image
- overprivileged registry token

canonical منابع

- <https://docs.docker.com/reference/cli/docker/> - Docker CLI reference [CLI]
- <https://docs.docker.com/engine/swarm/> - Docker Swarm mode [SWARM]
- <https://docs.docker.com/build/> - Docker Build manual [BUILD]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

PKI و DKR.37 – Swarm Architecture: Nodes, Services, Tasks, Raft

Advanced :Depth

DKR.23, DKR.30 :Prerequisites

پوشش اجباری

- replicated/global modes و Swarm init/join, managers/workers, desired state, services/tasks
- placement و scheduler constraints/preferences/resources
- state storage و Raft consensus/quorum, manager fault tolerance
- join tokens و mutual TLS/CA rotation, node certificates
- control-plane vs data-plane و service discovery/VIP/DNSRR

Labs / Experiments

- ۳-node swarm آزمایشی بساز؛ manager loss/quorum را simulate کن.

Failure scenarios

- lost quorum
- bad placement constraints
- cert/join problems

canonical منابع

- <https://docs.docker.com/engine/swarm/> - Docker Swarm mode [SWARM]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

و DKR.38 – Swarm Operations: Overlay, Routing Mesh, Updates, Secrets

DR

Advanced :Depth

DKR.37, DKR.26 :Prerequisites

پوشش اجباری

- endpoint modes و ingress/routing mesh, overlay networks, published ports

- update config و rolling updates/rollback, health/failure handling
- stacks, Compose compatibility limits, configs/secrets
- disaster recovery و backup/restore manager state
- relation to Kubernetes learning path :when NOT to use Swarm

Labs / Experiments

- manager backup/restore lab :rollback و service rolling update

Failure scenarios

- routing mesh unreachable
- update stuck
- quorum recovery

canonical منابع

- <https://docs.docker.com/engine/swarm/> - Docker Swarm mode [SWARM]
- <https://docs.docker.com/reference/compose-file/> - Compose file reference / Compose Specification implementation [COMPOSE]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Extension APIs و DKR.39 - Docker Engine Plugin System

Depth: Advanced+Contributor
Prerequisites: DKR.03, DKR.30, DKR.33

پوشش اجباری

- managed plugin lifecycle: install/create/enable/disable/inspect/set/upgrade/remove/push
- distribution به صورت image-like artifact و permission grant model
- plugin config.json, declared interfaces, mounts/devices/network/capabilities, Engine و rootfs, managed isolation
- extension interfaces و use cases: volume, network, logging و authorization؛ تفاوت plugin interface با built-in driver
- /legacy out-of-process Plugin API: discovery, activation handshake, socket/systemd activation, retry و migration/history و error semantics

- plugin debugging, dockerd log correlation, privilege/security review, availability/failure blast radius و platform limitations.
- developing a minimal plugin و تشخیص اینکه چه زمانی plugin مناسب است و چه زمانی external service /external service CSI/CNI/other extension model بهتر است.

Labs / Experiments

- یک managed plugin موجود را install/inspect/disable/enable کن و permission surface آن را تحلیل کن؛ سپس یک minimal test plugin یا mock Plugin API handshake بساز.

Failure scenarios

- plugin permission mismatch
- plugin fails activation
- volume/network depends on unavailable plugin
- unsafe privilege request
- legacy discovery/socket failure.

canonical منابع

- <https://docs.docker.com/engine/extend/> - Docker Engine managed plugin system
- https://docs.docker.com/engine/extend/plugin_api/ - Docker Plugin API
- https://docs.docker.com/engine/extend/legacy_plugins/ - Docker legacy plugins

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P4 Gate

- Gate P4: باید بتوانی Compose app پیچیده را reason کنی، daemon را امن اداره کنی، security و observability controls را دفاع کنی و Swarm failure/quorum/update را توضیح بدهی.

Container زیر P5 - Linux internals

اینجا Docker abstraction را می‌شکنیم. هدف این است که container را بدون Docker primitives سیستم عامل بازسازی کنی.

file descriptors و LNX.01 - Linux Process Model، syscalls، /proc

Internals :Depth

DKR.06 :Prerequisites

پوشش اجباری

- .exit و process/thread distinction، PID/TID، parent/child، fork/clone/clone3، execve، wait
- .credentials، environment، cwd/root، file descriptor table، pipes/sockets/TTY
- .PID 1 special behavior و signals، process groups/sessions
- .proc process introspection: status، cmdline، fd، ns، cgroup، mounts، limits/
- .troubleshooting به‌عنوان ابزار reasoning نه صرفاً

Labs / Experiments

- یک docker run/exec را با strace سطح مناسب روی runtime/child process مشاهده کن.

منابع canonical

<https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

LNX.02 - Linux Namespaces: همه انواع و API

Internals :Depth

LNX.01 :Prerequisites

پوشش اجباری

- namespace abstraction و lifetime/pinning در /proc/*/ns/

- namespaces: time, user, cgroup, IPC, UTS, network, mount, PID؛ scope و چیزهایی که isolate می‌کنند / نمی‌کنند.
- clone/clone3 flags و permission model.
- nested namespaces و parent relationships؛ visibility rules.
- nsenter/lsns و namespace debugging.

Labs / Experiments

- با unshare چند namespace بساز و با nsenter واردشان شو؛ بدون Docker hostname/PID/mount isolation بساز.

Failure scenarios

- CAP_SYS_ADMIN requirement
- mount propagation surprises

canonical منابع

- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path trace شود.

ID Mapping Internals و LNX.03 – User Namespace

Internals :Depth

LNx.02 :Prerequisites

پوشش اجباری

- subordinate IDs، setgroups restrictions، uid_map/gid_map و newuidmap/newgidmap.
- capabilities inside user namespace و scope آنها.
- idmapped mounts awareness و ownership problems در container storage.
- rootless networking/storage limitations از دید kernel privilege.
- security boundary caveats و kernel dependency.

Labs / Experiments

- user namespace بدون root بساز؛ UID 0 داخل را با host UID map کن.

منابع canonical

• <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS]

• <https://man7.org/linux/man-pages/man7/capabilities.7.html> - Linux capabilities(7) [LNX_CAP]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner. برای Internals/Contributor: حداقل یک spec/source؛ code path نیز trace شود.

delegation و LNX.04 - cgroup v2: hierarchy, controllers

Internals :Depth

LNX.01 :Prerequisites

پوشش اجباری

- single unified hierarchy, cgroup.procs, subtree_control, no-internal-process rule concepts
- controllers: cpu, memory, io, pids, cpuset و metrics/events
- OOM/memory events و PSI awareness و limits/weights/protectoins
- cgroup namespaces و delegation model و systemd integration
- mapping Docker resource flags to cgroup v2 files

Labs / Experiments

• بدون Docker cgroup بساز و memory/CPU/PIDs limits اعمال کن؛ OOM event را ثبت کن.

منابع canonical

• <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]

• https://docs.docker.com/engine/containers/resource_constraints/ - Docker container resource constraints [RESOURCE]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner. برای Internals/Contributor: حداقل یک spec/source؛ code path نیز trace شود.

LNK.05 - Mounts, VFS, rootfs, chroot و pivot_root

Internals :Depth

LNK.01, LNK.02 :Prerequisites

پوشش اجباری

- VFS و mount tree ؛ mount namespace و bind mounts .
- container implications و mount propagation: shared/slave/private/unbindable .
- pivot_root semantics و root filesystem assembly, chroot limitation .
- container pseudo-file systems و proc/sys/dev mounts داخل .
- device nodes basics و masked/readonly paths .

Labs / Experiments

- mount namespace بساز و rootfs را با pivot_root/chroot مقایسه کن.

Failure scenarios

- mount leak due propagation .
- missing /proc breaks tools .

منابع canonical

- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNK_NS]
- <https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

LNK.06 - OverlayFS Internals

Internals :Depth

LNK.05, DKR.21 :Prerequisites

پوشش اجباری

- layer stacking و lowerdir/upperdir/workdir/merged .
- copy-up, whiteouts, opaque directories, rename/hardlink/xattr caveats .

- performance impact awareness و page cache/inode behavior
- OverlayFS در classic Docker vs containerd overlayfs snapshotter conceptual mapping
- disk/inode pressure و backing filesystems روی failure modes

Labs / Experiments

- manual overlay mount بساز؛ delete/modify/rename را انجام بده و inspect را upper whiteouts/copy-up کن.

منابع canonical

<https://docs.kernel.org/filesystems/overlayfs.html> - Linux kernel - OverlayFS [LNX_OVERLAY] •

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Containers برای LNX.07 - Linux Networking Fundamentals

Foundation->Internals :Depth

LNK.01 :Prerequisites

پوشش اجباری

- MTU و Ethernet/L2، IP/L3، ARP/NDP، routes، gateways، interfaces
- localhost/interface semantics و sockets، TCP/UDP، listen/bind/connect
- sysctl و Linux routing tables/rules، IP forwarding
- local stub resolvers و DNS resolver path، /etc/resolv.conf/nsswitch
- network diagnostic mental model: packet path before tools

Labs / Experiments

- بدون Docker دو network namespace را با veth متصل و ping/TCP service برقرار کن.

منابع canonical

<https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNX_NS] •

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

nftables/iptables و LNX.08 – veth، Linux Bridge، NAT، conntrack

Internals :Depth

LNK.07 :Prerequisites

پوشش اجباری

- moving endpoint into namespace و veth peer mechanics
- gateway interface و Linux bridge FDB/L2 switching، bridge ports
- stateful conntrack و netfilter hooks/tables/chains، DNAT/SNAT/masquerade
- packet counters و rule inspection: iptables frontend/history vs nftables backend
- port publishing را کاملاً بدون Docker بازسازی کن.

Labs / Experiments

- یک "mini docker bridge" دستی: namespace+veth+bridge+route+NAT+port forward

Failure scenarios

- wrong forward policy
- conntrack stale
- asymmetric route

منابع canonical

- <https://docs.docker.com/engine/network/packet-filtering-firewalls/> - Docker packet filtering and firewalls [FIREWALL]
- <https://man7.org/linux/man-pages/man7/namespaces.7.html> - Linux namespaces(7) [LNK_NS]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

devices و LNX.09 – Linux Security: capabilities, seccomp, LSM

Internals :Depth

LNK.01, LNK.03 :Prerequisites

پوشش اجباری

- supplemental groups و DAC UID/GID/mode bits
- .namespaced capabilities و capability sets: permitted/effective/inheritable/bounding/ambient
- .default-deny/allow trade-offs و seccomp filter model, syscall surface
- .labels/profiles و Linux Security Modules: AppArmor/SELinux concepts
- .modern device control و devices, /dev, cgroup/device access history

Labs / Experiments

- minimal capability set برای یک task پیدا کن؛ seccomp profile سفارشی بساز.

Failure scenarios

- CAP_SYS_ADMIN overuse
- LSM denial misdiagnosed as permission bit

منابع canonical

- <https://man7.org/linux/man-pages/man7/capabilities.7.html> - Linux capabilities(7) [LNK_CAP]
- <https://man7.org/linux/man-pages/man2/seccomp.2.html> - Linux seccomp(2) [LNK_SECCOMP]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor حداقل یک spec/source code path نیز trace شود.

Container Host Integration و LNX.10 – systemd, cgroups

Internals+Ops :Depth

LNK.04, DKR.30 :Prerequisites

پوشش اجباری

- .cgroup ownership/delegation و systemd units/slices/scopes
- .restart/order/dependency semantics و docker.service/docker.socket

- daemon lifecycle during boot/shutdown و journal logging
- nested resource control و limits set by systemd vs Docker
- operational drift و host hardening, kernel/sysctl/module prerequisites

Labs / Experiments

- inspect را cgroup placement و dockerd unit overrides کن.

canonical منابع

- <https://docs.kernel.org/admin-guide/cgroup-v2.html> - Linux kernel - Control Group v2 [LNX_CGROUP]
- <https://docs.docker.com/reference/cli/dockerd/> - dockerd CLI reference [DOCKERD]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P5 Gate

- Gate P5: باید بدون Docker یک container-like process با namespaces+cgroup+rootfs+network بسازی و دقیقاً بگویی هر primitive چه isolation/resource property ایجاد کرده است.

runc و P6 - OCI, containerd

استاندارد و runtime stack را مستقیماً می‌خوانیم تا Docker دیگر جعبه سیاه نباشد.

OCI.01 - OCI Image Specification

Spec :Depth

DKR.09, LNX.06 :Prerequisites

پوشش اجباری

- Descriptor fields: mediaType, digest, size, annotations, platform/data/artifactType awareness
- image layout و Image manifest, image index, image config, filesystem layers
- rootfs ordering و layer changesets, whiteouts, DiffIDs
- canonicalization/extensibility و platform selection, annotations, conversion
- رابطه image artifact با runtime bundle

Labs / Experiments

- یک OCI layout دستی minimal بساز یا موجود را validate کن؛ descriptor graph را رسم کن.

منابع canonical

<https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

OCI.02 - OCI Runtime Specification

Spec :Depth

LNX.02, LNX.04, LNX.05, LNX.09 :Prerequisites

پوشش اجباری

- process/root/mounts/hostname/user/env/cwd و filesystem bundle, config.json
- Linux namespaces/resources/devices/capabilities/seccomp/sysctl/maskedPaths/readOnlyPaths

- platform-specific :create/start/kill/delete lifecycle و runtime state: creating/created/running/stopped (Linux additions و configuration sections (Windows/Solaris/FreeBSD awareness نسبت به در نسخه‌های جدید spec) بدون قاطی کردن semantics platformها.
- hooks و زمان/namespace اجرای آنها؛ annotations و features.
- distinction spec vs implementation و compliance language MUST/SHOULD/MAY.

Labs / Experiments

- runc spec تولید و config.json را field-by-field به kernel mechanisms map کن.

canonical منابع

<https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME] •

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

OCI.03 – OCI Distribution Specification

Spec :Depth

DKR.19, OCI.01 :Prerequisites

پوشش اجباری

- content discovery و conformance، pull/push workflows، blobs/manifests
- error codes و HTTP endpoints، monolithic/chunked uploads، digest verification
- unavailable-referrers compatibility behavior و referrers API
- registry-specific extensions boundaries و content management/delete behavior
- OCI distribution protocol با Docker Hub product features فرق

Labs / Experiments

- یک manifest/blob را فقط با curl pull کن؛ chunked upload کوچک اجرا کن.

canonical منابع

<https://specs.opencontainers.org/distribution-spec/> - OCI Distribution Specification [OCI_DIST] •

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

OCI Artifacts, Referrers و Non-image Content

Depth: Spec+Ecosystem
Prerequisites: OCI.01, OCI.03

پوشش اجباری

- artifactType/subject/referrers و graph of related artifacts.
- OCI distribution بر روی SBOM/provenance/signature/model/chart artifacts.
- interoperability considerations و media type design.
- چرا image index و generic artifacts برای modern supply chain مهم اند.
- Docker attestations, DHI, Helm/AI model packaging با relation.

Labs / Experiments

- referrers یک image دارای attestation را enumerate و graph کن.

منابع canonical

- <https://specs.opencontainers.org/image-spec/> - OCI Image Specification [OCI_IMAGE]
- <https://specs.opencontainers.org/distribution-spec/> - OCI Distribution Specification [OCI_DIST]
- <https://docs.docker.com/build/metadata/attestations/> - Docker build attestations [BUILD_ATTEST]
- <https://docs.docker.com/dhi/> - Docker Hardened Images [DHI]

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Plugin/Service Model و CTR.01 – containerd Architecture

Internals :Depth

OCI.01, OCI.02 :Prerequisites

پوشش اجباری

- namespaces (logical multi-tenancy) و containerd daemon، gRPC APIs، plugin architecture
- content, images, containers, tasks, snapshots, leases, events, introspection services
- clients: ctr, nerdctl, Docker/Moby, Kubernetes CRI به عنوان debug client نه UX production
- runtime/snapshotter selection و plugin discovery/configuration
- مرز containerd با image distribution و low-level runtime

Labs / Experiments

- inspect ctr namespaces/plugins/version کن و Docker objects را در namespace مربوطه پیدا کن.

canonical منابع

- <https://github.com/containerd/containerd> - containerd repository [CONTAINERD]
- <https://github.com/containerd/containerd/blob/main/docs/content-flow.md> - containerd content flow [CONTAINERD_FLOW]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back؛ study partner برای Internals/Contributor؛ حداقل یک spec/source code path نیز trace شود.

CTR.02 – containerd Content Flow، Images، Snapshots، Leases و GC

Internals :Depth

CTR.01 :Prerequisites

پوشش اجباری

- registry descriptor -> content store -> image metadata -> unpack -> committed snapshots -> active snapshot
- labels linking them و compressed blob vs uncompressed filesystem
- mount preparation و snapshotter concepts، parent chains
- why content may persist/disappear؛ garbage collection reachability و leases
- disk usage reasoning و Docker containerd image store mapping

Labs / Experiments

- با ctr pull/unpack؛ content ls/snapshot ls/images را correlate کن.

Failure scenarios

- orphaned content assumptions
- snapshot missing
- GC/lease surprises

canonical منابع

<https://github.com/containerd/containerd/blob/main/docs/content-flow.md> - containerd content flow [CONTAINERD_FLOW]

<https://docs.docker.com/engine/storage/containerd/> - containerd image store with Docker Engine [CONTAINERD_STORE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Shim و CTR.03 - containerd Containers vs Tasks, Runtime v2

Internals :Depth

CTR.01, OCI.02 :Prerequisites

پوشش اجباری

- task = live runtime process در برابر container metadata object.
- shim discovery/naming. runtime v2 architecture. ttRPC socket communication.
- 1:1 shim patterns vs grouped shim lifecycle؛ containerd daemon از container processes استقلال.
- shim -> runc create/start/kill/delete flow و stdio/reaping responsibilities.
- CRI boundary awareness و Kubernetes pod grouping labels بدون ورود کامل به Kubernetes.

Labs / Experiments

- docker container را با ctr/shim processes و sockets correlate کن؛ containerd restart behavior را observe کن.

Failure scenarios

- containerd restart vs running task
- shim crash
- .runtime not found

canonical منابع

<https://github.com/containerd/> - containerd runtime v2 architecture [CONTAINERD_RUNTIME] •
[containerd/blob/main/docs/runtime-v2.md](https://github.com/containerd/blob/main/docs/runtime-v2.md)

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Runtime Lifecycle و RNC.01 - runc CLI, OCI Bundle

Internals :Depth

OCI.02, CTR.03 :Prerequisites

پوشش اجباری

- .runc spec/run/create/start/state/list/exec/kill/delete/pause/resume/update/checkpoint/restore
- .higher-level setup برای separation create vs start و bundle/rootfs/config.json
- .user namespace requirements و rootless runc
- .runtime feature reporting و systemd cgroup driver awareness
- .why runc is low-level and not a full image/network manager

Labs / Experiments

- lifecycle؛ busybox rootfs + runc spec را بدون Docker/containerd اجرا کن.

Failure scenarios

- invalid config
- cgroup permission
- .rootfs/mount failure

canonical منابع

<https://github.com/opencontainers/runc> - runc repository / README [RUNC] •

<https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME] •

Definition of Done مشترک

• توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

CRIU و RNC.02 - libcontainer Internals, Bootstrap/Init

Implementation :Depth

RNC.01, LNX.02 :Prerequisites

پوشش اجباری

- native Go implementation و libcontainer factory/container/process abstractions.
- دو مرحله setns/clone sequencing و bootstrap/init، /proc/self/exe، nsenter
- .rootfs setup/mounts، namespaces، cgroups، capabilities، seccomp application ordering
- .sync protocol between parent/init و console/stdio
- .state transfer limitations و CRIU با checkpoint/restore

Labs / Experiments

• create/start code path libcontainer را trace کن؛ در صورت پشتیبانی checkpoint/restore آزمایشی.

canonical منابع

<https://github.com/opencontainers/runc/blob/main/> - runc libcontainer README [LIBCONTAINER] •

[libcontainer/README.md](#)

<https://github.com/opencontainers/runc> - runc repository / README [RUNC] •

Definition of Done مشترک

• توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P6 Gate

• Gate P6: باید OCI bundle را دستی اجرا کنی، containerd content/task model را توضیح بدهی و path shim->runc->kernel docs و process tree دفاع کنی.

Source Code Reading - P7 و ساخت اجزای مشابه

در این فاز هدف از «فهمیدن» به «توانایی دنبال کردن implementation و تغییر دادن آن» ارتقا پیدا می‌کند.

Test و SRC.MOB.01 - Moby Repository Architecture، Build

Contributor :Depth

DKR.03, CTR.03, RNC.01 :Prerequisites

پوشش اجباری

- /repo map: api/client/cmd/daemon/container/containerd/libnetwork/volume/images/graphdriver
- internal/integration و build tooling؛ از v29 به بعد تفاوت root module با github.com/moby/moby/v2 public
- github.com/moby/moby/client و github.com/moby/moby/api و release tags مستقل آن‌ها.
- build Moby from source در محیط ایزوله؛ awareness generated code/vendor/modules
- unit/integration test structure و test selection.
- daemon initialization/dependencies و subsystem ownership.
- security boundaries و contribution guidelines، issue reproduction.

Labs / Experiments

- Moby را build و یک test کوچک اجرا کن؛ daemon dependency map اولیه بکش.

منابع canonical

• <https://github.com/moby/moby> - Moby repository [MOBY]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

SRC.MOB.02 - Trace کردن Code Paths واقعی در Moby

Contributor :Depth

SRC.MOB.01 :Prerequisites

پوشش اجباری

- برای هر behavior مسیر را از CLI/API handler به daemon/service/storage/runtime دنبال کن.

- case studies اجباری: docker ps, docker run, docker exec, docker stop, docker logs, docker pull, docker network create, docker volume create.
- interface boundaries با containerd/libnetwork/volume/image services و error propagation.
- events/state persistence و locking/concurrency نقاط مهم.
- ثبت code map با file/function/commit hash؛ از line-by-line reading بی‌هدف اجتناب کن.

Labs / Experiments

- حداقل ۸ code-path report مستقل، هرکدام با sequence diagram.

canonical منابع

- <https://github.com/moby/moby> - Moby repository [MOBY]
- <https://docs.docker.com/reference/api/engine/> - Docker Engine API reference [API]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

API Mapping و SRC.CLI.01 - Docker CLI Source

Depth: Contributor

Prerequisites: DKR.03, META.DKR.03

پوشش اجباری

- API client creation و command tree/Cobra-style command construction, contexts/config/credentials.
- UX error handling و formatting/templates/progress streams.
- version negotiation و CLI command -> Engine API mapping.
- CLI plugins architecture برای Compose/Buildx/Model Runner و discovery boundaries.
- backward compatibility behavior و tests.

Labs / Experiments

- یک command ساده را از parser تا API request trace کن.

canonical منابع

- <https://github.com/docker/cli> - Docker CLI repository [DOCKER_CLI_REPO]
- <https://docs.docker.com/reference/api/engine/> - Docker Engine API reference [API]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Frontend و SRC.BLD.01 – BuildKit Source: Solver, Worker, Cache, Executor

Contributor: **Depth**

DKR.14, CTR.02: **Prerequisites**

پوشش اجباری

- BuildKit repo map: solver, worker, executor, snapshot, cache, source, frontend, exporter, session, client/llb
- solver scheduling/caching path و LLB graph ingestion
- containerd/OCI reuse و worker/executor/snapshotter interfaces
- Dockerfile frontend boundary و frontend/gateway
- content stores و cache records/GC/export/import

Labs / Experiments

- یک Dockerfile build را تا LLB op و solver/worker execution در source trace کن.

canonical منابع

- <https://github.com/moby/buildkit> - BuildKit repository [BUILDKIT_REPO]
- <https://docs.docker.com/build/buildkit/> - Docker BuildKit manual [BUILDKIT]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Compose Implementation Source و SRC.CMP.01 – Compose Specification

Contributor/Ecosystem: **Depth**

DKR.28: **Prerequisites**

پوشش اجباری

- Docker Compose Go implementation awareness و Compose Spec repository/model concepts

- parse -> interpolation -> normalization -> merge/include -> project model -> Engine API .orchestration
- .compatibility surface و CLI plugin integration
- تفاوت spec-defined behavior با implementation-specific extensions
- انتخاب یک issue کوچک برای trace یا test.

Labs / Experiments

- compose config path را در source trace کن.

canonical منابع

- <https://github.com/docker/compose> - Docker Compose repository [COMPOSE_REPO]
- <https://docs.docker.com/reference/compose-file/> - Compose file reference / Compose Specification implementation [COMPOSE]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

LAB.01 - ساخت Mini Container Runtime

Capstone :Depth

Prerequisites : LNX.02, LNX.04, LNX.05, LNX.08, LNX.09

پوشش اجباری

- نسخه ۱: process + chroot؛ نسخه ۲: mount/PID/UTS namespaces؛ نسخه ۳: cgroup limits؛ نسخه ۴: capabilities/seccomp.
- network namespace/veth/bridge؛ نسخه ۵: .capabilities/seccomp.
- .exec user process و rootfs preparation.
- .cleanup و parent-child synchronization.
- CLI ساده run/exec/kill/state awareness؛ لزوماً OCI-compliant نیست در ابتدا.
- نسخه اختیاری Go با الهام از libcontainer، بدون copy کردن implementation.

Labs / Experiments

- runtime خودت را برای busybox اجرا کن و با runc behavior مقایسه کن.

Failure scenarios

- cleanup leaked namespaces/mounts

- PID1/signal issue
- .cgroup permissions

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

LAB.02 - ساخت Mini OCI Image/Registry Explorer

Capstone :Depth

OCI.01, OCI.03, CTR.02 :Prerequisites

پوشش اجباری

- HTTP registry client برای manifest/index/blob و digest verification.
- blob download/decompress، platform manifest selection و layer application.
- OCI image layout writer/reader minimal.
- unpacker در whiteout handling به عنوان advanced extension.
- attestation/referrer discovery optional.

Labs / Experiments

- بدون Docker یک image کوچک را pull و rootfs materialize کن.

Definition of Done مشترک

• توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m: teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

LAB.03 - بازسازی Docker Bridge Networking بدون Docker

Capstone :Depth

LNx.08 :Prerequisites

پوشش اجباری

- namespace + veth + bridge + address + route + forwarding + NAT + DNS strategy.
- netfilter با host port forwarding.
- multiple containers و east-west communication.

• cleanup/idempotency script.

• مقایسه Docker user-defined bridge با packet/rule state.

Labs / Experiments

• اسکریپت کامل create/run/destroy برای mini-network بنویس.

Definition of Done مشترک

• توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P7 Gate

• Gate P7: حداقل یک component را از build source کرده‌ای، چند code path را trace کرده‌ای و runtime/network/image mechanisms را خودت بازسازی کرده‌ای.

P8 – Cross-platform، Docker Product Ecosystem

قابلیت‌های جدید

این فاز breadth محصولی را می‌دهد تا «Docker expert» فقط معادل Linux Engine expert نباشد؛ وزن این بخش کمتر از Core/Internals است و دائماً version-sensitive است.

Host Integration و PLT.01 – Docker Desktop Architecture

Product :Depth

DKR.02, LNX.10 :Prerequisites

پوشش اجباری

- چرا Linux containers روی macOS/Windows به Linux VM/kernel نیاز دارند؛ Desktop control plane و CLI integration.
- filesystem sharing، networking، DNS/localhost semantics و performance differences.
- VM resource allocation، proxy/cert integration و Kubernetes optional feature awareness.
- Enhanced Container Isolation concept و enterprise hardening boundaries.
- Desktop licensing/organization policy awareness بدون ورود business administration عمیق.

Labs / Experiments

- روی Desktop در صورت دسترسی، host/container filesystem/network behavior را با native Linux مقایسه کن.

منابع canonical

• <https://docs.docker.com/desktop/> – Docker Desktop manual [DESKTOP]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ teach-back به study partner برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Windows Containers و PLT.02 - Windows: WSL2، Hyper-V

Product :Depth

PLT.01 :Prerequisites

پوشش اجباری

- docker-desktop distribution، Linux filesystem performance best و WSL2 backend architecture .practices
- .security/isolation trade-offs و Hyper-V backend concept
- .Windows kernel/image compatibility و Linux containers vs Windows containers
- .unsupported flags awareness و Windows networking/storage/process differences
- .path/permission pitfalls و cross-platform Compose/build behavior

Labs / Experiments

- اگر Windows داری، WSL2 data location/performance و Linux-vs-Windows container mode را document کن.

canonical منابع

- <https://docs.docker.com/desktop/features/wsl/> - Docker Desktop WSL 2 backend [DESKTOP_WSL]
- <https://docs.docker.com/desktop/> - Docker Desktop manual [DESKTOP]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Product Surface به عنوان Hardened Images و PLT.03 - Docker Hub، Scout

Product :Depth

DKR.19, DKR.35 :Prerequisites

پوشش اجباری

- lifecycle policies و Hub repositories/orgs/teams/access/tokens/webhooks/trusted content/API .awareness
- .CI integration و Scout SBOM/vulnerability/policy workflows
- shared و DHI catalog/minimal images/provenance/VEX/signatures/compliance variants .responsibility

- فرق Docker-hosted product features با open standards capabilities
- deprecation/retirement tracking برای hosted features: docs/release notes source of truth

Labs / Experiments

- یک repo آزمایشی Hub + webhook یا Scout workflow در حد دسترسی موجود.

canonical منابع

- <https://docs.docker.com/docker-hub/> - Docker Hub manual [HUB]
- <https://docs.docker.com/scout/> - Docker Scout manual [SCOUT]
- <https://docs.docker.com/dhi/> - Docker Hardened Images [DHI]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

PLT.04 – Emerging Docker AI/Agent Surface: Model Runner, Sandboxes

MCP Toolkit

Awareness :Depth

OCI.04, PLT.01 :Prerequisites

پوشش اجباری

- Docker Model Runner: model artifacts via OCI, local inference engines. OpenAI/Ollama-compatible APIs و Compose integration.
- Docker Sandboxes: isolated microVM sandboxes برای daemon/filesystem/network coding agents, isolation.
- MCP Catalog/Toolkit/Gateway: containerized/remote MCP servers و policy-aware tool integration.
- این محصولات جزء core container runtime mastery نیستند؛ فقط architecture/use-case/security boundaries باید شناخته شوند.
- این module باید پیش از هر مطالعه دوباره against current docs audit شود چون surface سریع تغییر می‌کند.

Labs / Experiments

- در صورت نیاز شخصی: یک model یا MCP server کوچک با Docker product workflow اجرا کن.

canonical منابع

- <https://docs.docker.com/ai/model-runner/> - Docker Model Runner [MODEL]
- <https://docs.docker.com/ai/sandboxes/> - Docker Sandboxes [SANDBOX]
- <https://docs.docker.com/ai/mcp-catalog-and-toolkit/> - Docker MCP Catalog and Toolkit [MCP]

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P8 Gate

- Gate P8: می‌توانی توضیح بدهی کدام بخش Docker استاندارد/open-source core است و کدام بخش /hosted platform differences؛ Desktop/product feature را بدون قاطی کردن abstractionها بیان می‌کنی.

،P9 – Production، Failure Engineering، Ecosystem Interview و Contribution

دانش را تحت فشار production، failure، design trade-offs و مصاحبه تثبیت می‌کند و آن را به contribution واقعی وصل می‌کند.

Lifecycle Management و OPS.01 – Production Engineering

Production :Depth

DKR.30, DKR.32, DKR.35 :Prerequisites

پوشش اجباری

- .resource governance و host provisioning/hardening، upgrade/rollback، capacity planning
- .image lifecycle: base updates، rebuild cadence، registry retention، vulnerability response
- .stateless/stateful design و data backup/restore، daemon/containerd data، volume DR
- .incident evidence preservation و logging/metrics/alerts، SLO-oriented observability
- .change management و CI/CD integration، immutable delivery، secrets، rollback

Labs / Experiments

- یک production runbook برای host+registry+stateful app تهیه و disaster drill اجرا کن.

canonical منابع

- <https://docs.docker.com/engine/> - Docker Engine manual [ENGINE]
- <https://docs.docker.com/engine/release-notes/29/> - Docker Engine v29 release notes [RELEASE29]
- <https://docs.docker.com/scout/> - Docker Scout manual [SCOUT]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

OPS.02 – Performance Engineering

Production :Depth

DKR.07, DKR.21, DKR.24 :Prerequisites

پوشش اجباری

- syscall overhead و CPU throttling/scheduling, memory pressure/page cache/OOM, PIDs
- IO benchmarks و storage CoW/copy-up, bind/volume/filesystem choices
- connection scaling و network NAT/overlay/encryption/MTU overhead
- lazy snapshotter awareness :image pull/decompression/startup latency
- CI build optimization و BuildKit parallelism/cache/context transfer

Labs / Experiments

- baseline benchmark بساز و یک bottleneck CPU/memory/storage/network/build را isolate کن.

Failure scenarios

- benchmark noise
- host contention misattributed to container

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m به teach-back study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Troubleshooting Matrix و OPS.03 – Failure Engineering

Production :Depth

OPS.01 :Prerequisites

پوشش اجباری

- incident categories: daemon, image, pull/auth, build, process, signal, resource, storage, network, DNS, firewall, permissions, security policy, runtime/containerd/runc
- فرضیه محور -> root cause -> experiment -> evidence -> boundary -> debugging: symptoms -> prevention
- خطاها را عمداً بساز: /disk full/inodes, OOM, DNS, MTU, bad routes, permission/UID, seccomp, capability, daemon config, registry auth, corrupt/unsupported images

- test to prevent و postmortem format: trigger, contributing factors, detection gap, remediation .regression
- chaos فقط در lab؛ production safety rules.

Labs / Experiments

- حداقل ۲۰ incident کارت با reproducible setup و expected diagnosis بساز.

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Boundary Reasoning و OPS.04 – Ecosystem Comparison

Advanced :Depth

CTR.03, RNC.01 :Prerequisites

پوشش اجباری

- Docker/Moby/containerd/runc/nerdctl و layers جایگزین شده توسط هرکدام.
- Podman/Buildah/Skopeo، CRI-O و daemonless/rootless trade-offs.
- Kubernetes CRI و جایگاه containerd/CRI-O؛ چرا Docker Engine مستقیماً Kubernetes runtime امروز نیست.
- gVisor/Kata/microVM sandbox concepts و stronger isolation trade-offs.
- Wasm/OCI artifacts و nontraditional workloads awareness؛ انتخاب tool بر اساس brand نه problem.

Labs / Experiments

- یک جدول architecture replacement بساز و یک workload را با Docker و Podman/nerdctl مقایسه کن.

canonical منابع

- <https://github.com/containerd/containerd> - containerd repository [CONTAINERD]
- <https://github.com/opencontainers/runc> - runc repository / README [RUNC]
- <https://specs.opencontainers.org/runtime-spec/> - OCI Runtime Specification [OCI_RUNTIME]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Merged PR تا Issue :CONTRIB.01 – Contribution Workflow

Contributor :Depth

SRC.MOB.01, SRC.BLD.01 :Prerequisites

پوشش اجباری

- انتخاب project: moby/moby, docker/cli, moby/buildkit, containerd/containerd, opencontainers/runc, OCI specs, docker/compose
- maintainers expectations و issue triage, reproduction, minimal test case, reading contributing docs
- شروع از docs/test/error-message/small bug؛ سپس behavior changes/features
- backward compatibility و commit hygiene, tests, CI, review iteration
- ثبت learning report: چه subsystem را فهمیدی، چه assumptions غلط بود، چه code paths یاد گرفتی.

Labs / Experiments

- حداقل یک issue reproduction عمومی و یک PR واقعی یا patch قابل ارسال.

canonical منابع

- <https://github.com/moby/moby> – Moby repository [MOBY]
- <https://github.com/moby/buildkit> – BuildKit repository [BUILDKIT_REPO]
- <https://github.com/containerd/containerd> – containerd repository [CONTAINERD]
- <https://github.com/opencontainers/runc> – runc repository / README [RUNC]
- <https://github.com/docker/compose> – Docker Compose repository [COMPOSE_REPO]

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

INT.01 – Interview Mastery: 30 seconds / 3 minutes / 30 minutes

Interview :Depth

OPS.03 :Prerequisites

پوشش اجباری

- برای هر concept سه depth پاسخ؛ شروع concise و deep dive فقط وقتی interviewer فضا می‌دهد.
- conceptual questions: container vs VM, image, namespaces/cgroups, networking, storage, BuildKit, OCI, containerd/runc

- trade-offs و operational questions: outage/debugging, security, resource limits, deployment design
- whiteboard architecture و packet/process/code-path drawing از حافظه.
- هدف: نشان دادن مدل ذهنی و reasoning, نه نمایش superiority نسبت به interviewer.

Labs / Experiments

- بانک حداقل ۲۰۰ سؤال tiered; mock interview هفتگی با study partner.

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Production Scenario Drills و INT.02 – Adversarial Interview

Interview :Depth

INT.01 :Prerequisites

پوشش اجباری

- interviewer فقط symptom می‌دهد و candidate باید سؤال مناسب برای evidence بپرسد.
- cross-layer scenarios که root cause در Linux ولی symptom در Docker است.
- /design trade-off: bind vs volume, rootless vs rootful, bridge vs host, Compose vs Swarm
- Kubernetes, distroless vs debugability
- false premise detection: سؤال مصاحبه‌ای که premise آن ناقص/غلط است را محترمانه اصلاح کن.
- post-answer self-review: چه assertionهایی بدون evidence یا version caveat بودند؟

Labs / Experiments

- هر هفته roles را عوض کنید: interviewer/candidate/observer.

Definition of Done مشترک

- توضیح بدون note + diagram از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

Re-certification و CAP.01 – Final Docker Mastery Capstone

Capstone :Depth

CONTRIB.01, INT.02 :Prerequisites

پوشش اجباری

- از Docker CLI تا Docker API/Moby/containerd/shim/runc/kernel یک docker run را از حافظه diagram کن و سپس source-verify.
- image را از registry protocol تا explain content store/snapshot/rootfs/runtime و manually inspect کن.
- bridge packet path و storage copy-up path را با kernel primitives explain و reproduce کن.
- یک incident ناشناخته را time-boxed debug کن؛ یک code change را locate و patch prototype کن.
- هر ۶-۱۲ ماه یا پس از selected gates، major version را re-run و syllabus coverage را re-audit کن.

Labs / Experiments

- دو روز capstone: build+run+break+debug+source trace+teach-back+mock interview.

Definition of Done مشترک

- توضیح بدون diagram + note از حافظه؛ اجرای lab؛ ایجاد حداقل یک failure؛ توضیح ارتباط با modules مجاور؛ پاسخ 30s/3m/30m؛ teach-back به study partner. برای Internals/Contributor: حداقل یک spec/source code path نیز trace شود.

P9 Gate

- Gate P9 / Graduation: mastery evidence شامل code-path reports، incident reports، labs، حداقل یک contribution attempt و توانایی تدریس/مصاحبه در چند سطح باشد.

قرارداد Master Prompt و Visual QA برای همه PDFها

- فایل canonical: `meta/prompts/MASTER_PDF_PROMPT.md`. نسخه اصلی ۶۴ بندی کاربر نیز بدون حذف در `meta/prompts/source/REPORT_GENERATION_PROMPT_ORIGINAL.md` نگهداری می‌شود.
- precedence: instruction همان checklist -> Deep Study overrides -> document اصلی. override اصلی این پروژه: **Vazirmatn** به جای مثال‌های فونت دیگر.
- هر candidate PDF باید 180-220 DPI + render + font embedding check + structural preflight داشته باشد.
- تمام صفحات باید بصری بازبینی شوند؛ spot-check برای status نهایی پذیرفته نیست.
- defect workflow: `source fix -> rebuild -> rerender -> full review`. اصلاح مستقیم artifact بدون sync source ممنوع است.
- visual review شامل cover, TOC true-RTL, page numbers, borders/margins, headings/orphans, tables, suspicious blank pages و boxes, code/Latin bidi, citations/URLs, glyphs است.
- هر revision از آخرین canonical source ساخته می‌شود و regression fixes قبلی باید حفظ شوند.
- هر PDF باید source baseline, research cutoff, Document ID, matching source version و Visual QA status/date داشته باشد.

استاندارد PDF های بعدی

- 1. Document metadata و version baseline
- 1. Scope Contract: In Scope / Out of Scope / Prerequisites
- 1. Definition of Done و Learning objectives
- 1. Why this exists + historical context (فقط به اندازه فهم design)
- 1. Terminology و core mental model
- 1. Architecture / data flow / state machine diagrams
- 1. Detailed mechanics و configuration surface
- 1. Internals و underlying Linux/OCI mechanisms
- 1. Implementation/source-code path (اگر مربوط است)
- 1. Connections به modules دیگر و alternative tools
- 1. Hands-on labs و prediction-before-execution questions
- 1. Failure modes و troubleshooting decision tree
- 1. Production/security/performance considerations
- 1. Common misconceptions و version caveats
- 1. Interview ladder: 30s / 3m / 30m
- 1. Teach-back questions برای study partner
- 1. Mastery checklist و evidence required
- 1. References با provenance و source version/commit در صورت نیاز
- 1. Open questions و changelog
- 1. Typography/layout: Vazirmatn، RTL، واقعی، LTR isolation، border، margins، A4، code/IDs، Persian prose numerals و footer
- 1. Full visual QA report: همه صفحات render/inspect؛ defects و iteration تا PASS کامل

روش سیستماتیک استخراج Syllabus از منابع معتبر

1. Canonical documentation inventory

navigation رسمی docs را flatten کن و تمام manuals/reference/product sections را inventory کن. این inventory scope موجود را می‌گوید، نه ترتیب آموزش.

2. Book TOC extraction

TOC کتاب‌های معتبر را برای pedagogical ordering و مثال‌ها بگیر؛ کتاب به تنهایی معیار completeness یا currency نیست.

3. Standards/specifications

هر spec رسمی را section-by-section وارد union کن. موضوعی که در spec هست ولی کتاب ندارد = syllabus gap بالقوه.

4. Reference surface audit

CLI/API/file-format/daemon references را برای capability coverage بررسی کن؛ همه flags حفظ نمی‌شوند ولی وجود هر capability باید شناخته شود.

5. Source-tree audit

دایرکتوری‌ها/subsystems اصلی source را inventory کن. subsystem بزرگ بدون معادل در syllabus = gap یا deliberate out-of-scope که باید ثبت شود.

6. Dependency extraction

برای هر topic prerequisites مفهومی استخراج کن (مثلاً namespaces/veth/bridge/routing -> Docker bridge /NAT/firewall).

7. Release/deprecation audit

release notes، deprecated/removed/experimental features را بررسی کن تا syllabus با current behavior sync بماند.

8. Union + normalization

topic های هم نام را merge کن ولی provenance را نگه دار؛ هیچ topic فقط چون یک منبع دیگر آن را ندارد حذف نشود.

9. Classification

هر topic را Foundation/Core/Production/Internals/Spec/Implementation/Product/Historical دسته بندی کن.

10. Dependency DAG -> Spiral Order

اول DAG بساز، سپس order آموزشی spiral طراحی کن؛ prerequisite را کامل و خطی قبل از موضوع اصلی تمام نکن.

11. Coverage Matrix

برای هر topic ستون های Book/Docs/Spec/Source/Lab/Failure/Interview/PDF/Done نگه دار.

12. Adversarial gap review

از AI و انسان بخواهند مشخصاً «چه چیز مهمی حذف شده؟ چه subsystem ی source دارد ولی syllabus ندارد؟» نه اینکه roadmap جدید از حافظه بسازند.

Protocol مطالعه دونفره

- Weekly Teach Without Notes: یک نفر ۲۰-۳۰ دقیقه موضوع را از حافظه روی تخته توضیح می‌دهد؛ نفر دوم فقط سؤال boundary و counterexample می‌پرسد.
- Attack/Defend: claim: هایی مثل "containers are lightweight VMs" یا "namespaces provide security" را یکی دفاع و دیگری نقد می‌کند؛ سپس منبع canonical بررسی می‌شود.
- Mock Incident: interviewer فقط symptom و evidence را مرحله‌ای می‌دهد؛ debugger باید سؤال درست بپرسد، نه command تصادفی بزند.
- Role Rotation: teacher / interviewer / candidate / reviewer عوض شود تا skill توضیح دادن و skill کشف gap هر دو رشد کنند.
- Error Ledger: هر برداشت غلطی که کشف می‌شود با "I assumed X; actual model is Y; evidence: source/lab" ثبت شود.

ساختار پیشنهادی Repository

- `deep-study/README.md` - ورودی پروژه و هدف
- `deep-study/PROJECT.md` - mission و scope بلندمدت
- `deep-study/STUDY_METHOD.md` - AI usage و spiral، labs، teach-back روش
- `deep-study/CONTENT_STANDARD.md` - template اجباری PDFها
- `deep-study/SOURCE_POLICY.md` - hierarchy و copyright policy منابع
- `deep-study/RESEARCH_METHOD.md` - از منابع syllabus/module الگوریتم استخراج
- `deep-study/MASTERY.md` - Definition of Done و gates
- `deep-study/AGENTS.md` - های آینده ChatGPT/Codex/agentدستورالعمل برای
- `deep-study/meta/prompts/` - syllabus research, gap analysis, module research, source verification, PDF generation, interview, lab, code trace, update
- `deep-study/curriculum/devops/docker/` - syllabus.md/pdf, coverage-matrix, dependency-map
- `deep-study/subjects/docker/` - source Markdown هر module و supporting labs/diagrams
- `deep-study/pdf/docker/` - مطالعه generated های PDF
- `deep-study/library/catalog.*` - metadata وضعیت و لینک ownership فایل اگر commercial book کتابها؛ فقط اگر
حق نگهداری اجازه دهد/ license

Source Policy و کتاب‌ها

- Source of Truth فنی: official docs/specs/source code. کتاب برای coherence و pedagogical order است، نه current truth.
- کتاب اصلی پیشنهادی: Docker: Up & Running, 3rd Edition؛ companion: Docker Deep Dive, 4th Edition. هر دو با docs/specs فعلی reconcile می‌شوند.
- فایل کتاب‌های تجاری داخل Git repo قرار نگیرد مگر license/حق استفاده صراحتاً اجازه دهد. در repo catalog metadata، ownership status، edition و لینک قانونی نگهداری شود.
- منابع open-source/specifications بر اساس license خودشان مدیریت و در صورت vendor کردن، /license/commit/version ثبت شود.

Coverage Matrix Schema

Mastere d	Version reviewed	Intervie w	Failur e	La b	Source	Spec	Doc s	Boo k	Study PDF	Topic ID
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	DKR.09
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	DKR.24
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	LNK.04
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	OCI.02
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	CTR.03
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	RNC.02
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	SRC.MOB. 02
	2026-08-11	بله	بله	بله	as applicable	as applicable	بله	بله	planned	OPS.03

Mastery Checklist عمومی

- می‌توانم مفهوم را بدون jargon اضافی در 30 ثانیه تعریف کنم.
- می‌توانم در 3 دقیقه mechanism و trade-off اصلی را توضیح بدهم.
- می‌توانم در 30 دقیقه failure modes، internals، connections و deep-dive را کنم.
- می‌توانم architecture/state/data flow را از حافظه رسم کنم.
- می‌توانم capability یا flag دقیق را از reference پیدا کنم بدون تکیه به حفظ.
- حداقل یک happy-path lab و یک break/fix lab اجرا کرده‌ام.
- می‌توانم common misconceptions و version caveats را بگویم.
- اگر module internals است، spec/source code را مستقیم خوانده و evidence ثبت کرده‌ام.
- می‌توانم موضوع را به نفر دوم تدریس کنم و به counterexample‌های او پاسخ بدهم.
- Open questions باقی‌مانده صریحاً ثبت شده‌اند؛ "فهمیدم" جای سؤال حل‌نشده را نمی‌گیرد.

Syllabus Prompt pattern برای استخراج/به‌روزرسانی

I am maintaining an expert-level Docker syllabus.

Use ONLY the supplied canonical documentation inventory, specification TOCs, source-tree inventory, release/deprecation notes, and selected book TOCs as evidence.

- 1) Build the UNION of topics without dropping topics that appear in only one authoritative source.
- 2) Preserve provenance for every topic.
- 3) Classify each topic: Foundation / Core / Production / Internals / Spec / Implementation / Product / Historical.
- 4) Extract prerequisites and build a dependency DAG.
- 5) Identify gaps: docs-without-syllabus, spec-without-syllabus, major source subsystem-without-syllabus, reference capability-without-syllabus.
- 6) Mark version-sensitive, deprecated, experimental, and platform-specific topics.
- 7) Do NOT generate study order until the inventory and gap review are complete.
- 8) After approval, turn the DAG into a spiral study order and propose labs/failure/source-reading evidence for each module.

Bibliography / Canonical Source Registry

- <https://docs.docker.com/> :Docker Documentation - **DOCS** •
- <https://docs.docker.com/engine/> :Docker Engine manual - **ENGINE** •
- <https://docs.docker.com/engine/release-notes/29/> :Docker Engine v29 release notes - **RELEASE29** •
- <https://docs.docker.com/reference/api/engine/> :Docker Engine API reference - **API** •
- <https://docs.docker.com/reference/cli/docker/> :Docker CLI reference - **CLI** •
- <https://docs.docker.com/engine/manage-resources/contexts/> :Docker contexts - **CONTEXTS** •
- <https://docs.docker.com/engine/extend/> :Docker Engine managed plugins - **PLUGINS** •
- https://docs.docker.com/engine/extend/plugin_api/ :Docker Plugin API - **PLUGIN_API** •
- <https://docs.docker.com/reference/cli/dockerd/> :dockerd CLI reference - **DOCKERD** •
- <https://docs.docker.com/reference/dockerfile/> :Dockerfile reference - **DOCKERFILE** •
- <https://docs.docker.com/build/> :Docker Build manual - **BUILD** •
- <https://docs.docker.com/build/buildkit/> :Docker BuildKit manual - **BUILDKIT** •
- <https://docs.docker.com/build/metadata/attestations/> :Docker build attestations - **BUILD_ATTEST** •
- <https://docs.docker.com/build/building/cdi/> :Docker Build CDI - **CDI** •
- <https://docs.docker.com/compose/> :Compose file reference / Compose Specification implementation - **COMPOSE** •
docs.docker.com/reference/compose-file/
- <https://docs.docker.com/engine/storage/> :Docker Engine storage - **STORAGE** •
- <https://docs.docker.com/engine/storage/containerd/> :containerd image store with Docker Engine - **CONTAINERD_STORE** •
- <https://docs.docker.com/engine/network/> :Docker Engine networking - **NETWORK** •
- <https://docs.docker.com/engine/network/packet-filtering-firewalls/> :Docker packet filtering and firewalls - **FIREWALL** •
- <https://docs.docker.com/engine/containers/resource-constraints/> :Docker container resource constraints - **RESOURCE** •
- <https://docs.docker.com/engine/containers/gpu/> :Docker Engine GPU access - **GPU** •
- <https://docs.docker.com/engine/security/rootless/> :Docker rootless mode - **ROOTLESS** •
- <https://docs.docker.com/engine/security/userns-remap/> :Docker user namespace remapping - **USERNS** •
- <https://docs.docker.com/engine/swarm/> :Docker Swarm mode - **SWARM** •
- <https://docs.docker.com/desktop/> :Docker Desktop manual - **DESKTOP** •
- <https://docs.docker.com/desktop/features/wsl/> :Docker Desktop WSL 2 backend - **DESKTOP_WSL** •
- <https://docs.docker.com/docker-hub/> :Docker Hub manual - **HUB** •
- <https://docs.docker.com/scout/> :Docker Scout manual - **SCOUT** •

<https://docs.docker.com/dhi/> :Docker Hardened Images - **DHI** •

<https://docs.docker.com/ai/model-runner/> :Docker Model Runner - **MODEL** •

<https://docs.docker.com/ai/sandboxes/> :Docker Sandboxes - **SANDBOX** •

<https://docs.docker.com/ai/mcp-catalog-and-toolkit/> :Docker MCP Catalog and Toolkit - **MCP** •

<https://specs.opencontainers.org/image-spec/> :OCI Image Specification - **OCI_IMAGE** •

<https://specs.opencontainers.org/runtime-spec/> :OCI Runtime Specification - **OCI_RUNTIME** •

<https://specs.opencontainers.org/distribution-spec/> :OCI Distribution Specification - **OCI_DIST** •

<https://man7.org/linux/man-pages/man7/namespaces.7.html> :Linux namespaces(7) - **LNK_NS** •

[https://docs.kernel.org/admin-guide/cgroup-](https://docs.kernel.org/admin-guide/cgroup-v2.html) :Linux kernel - Control Group v2 - **LNK_CGROUP** •
[v2.html](https://docs.kernel.org/admin-guide/cgroup-v2.html)

<https://docs.kernel.org/filesystems/overlayfs.html> :Linux kernel - OverlayFS - **LNK_OVERLAY** •

<https://man7.org/linux/man-pages/man7/capabilities.7.html> :Linux capabilities(7) - **LNK_CAP** •

<https://man7.org/linux/man-pages/man2/seccomp.2.html> :Linux seccomp(2) - **LNK_SECCOMP** •

[https://github.com/containerd/containerd/](https://github.com/containerd/containerd/blob/main/docs/runtime-v2.md) :containerd runtime v2 architecture - **CONTAINERD_RUNTIME** •
[containerd/blob/main/docs/runtime-v2.md](https://github.com/containerd/containerd/blob/main/docs/runtime-v2.md)

[https://github.com/containerd/containerd/blob/](https://github.com/containerd/containerd/blob/main/docs/content-flow.md) :containerd content flow - **CONTAINERD_FLOW** •
[main/docs/content-flow.md](https://github.com/containerd/containerd/blob/main/docs/content-flow.md)

<https://github.com/containerd/containerd> :containerd repository - **CONTAINERD** •

<https://github.com/opencontainers/runc> :runc repository / README - **RUNC** •

[https://github.com/opencontainers/runc/blob/main/](https://github.com/opencontainers/runc/blob/main/libcontainer/README.md) :runc libcontainer README - **LIBCONTAINER** •
[libcontainer/README.md](https://github.com/opencontainers/runc/blob/main/libcontainer/README.md)

<https://github.com/moby/moby> :Moby repository - **MOBY** •

<https://github.com/docker/cli> :Docker CLI repository - **DOCKER_CLI_REPO** •

<https://github.com/docker/buildkit> :BuildKit repository - **BUILDKIT_REPO** •

<https://github.com/docker/compose> :Docker Compose repository - **COMPOSE_REPO** •

[https://www.oreilly.com/](https://www.oreilly.com/library/view/docker-up/9781098131814/) :Docker: Up & Running, 3rd ed. - Sean P. Kane, Karl Matthias - **BOOK_UR** •
[library/view/docker-up/9781098131814/](https://www.oreilly.com/library/view/docker-up/9781098131814/)

[https://www.packtpub.com/en-us/product/](https://www.packtpub.com/en-us/product/docker-deep-dive-9781837028344) :Docker Deep Dive, 4th ed. - Nigel Poulton - **BOOK_DD** •
[docker-deep-dive-9781837028344](https://www.packtpub.com/en-us/product/docker-deep-dive-9781837028344)

Change Policy

- Document versioning: MAJOR برای scope-breaking، MINOR برای coverage/source-baseline مهم، PATCH برای factual/layout/citation fix محدود.
- هر تغییر foundational باید downstream impact list داشته باشد و PDF های potentially invalidated را مشخص کند.
- هر تغییر syllabus باید دلیل، منبع، affected IDs و تاریخ review داشته باشد. حذف topic فقط با deliberate out-of-scope rationale مجاز است.
- قبل از ساخت PDF هر module، همان module دوباره current-audit می‌شود؛ این syllabus نقشه است، نه جایگزین verification نسخه فعلی.

Changelog - v1.1.0

- Master PDF Prompt / Vazirmatn / full-page visual QA contract به syllabus اضافه شد.
- research cutoff به 11-08-2026 ارتقا یافت و latest verification rule از search snippet به canonical-page verification سخت‌گیرانه‌تر شد.
- current baseline روی Docker Engine 29.7.2 و OCI Runtime 1.3.0 / Image 1.1.1 / Distribution 1.1.1 تثبیت شد.
- version-sensitive delta های Engine 29.x (containerd image store, embedded-containerd experimental map به image mounts, cgroup v1 deprecation, rootless default changes, Moby Go module changes اضافه شدند.
- DKR.39 برای managed/legacy Docker Engine Plugin System و extension APIs اضافه شد.
- coverage موضوعات BuildKit GC/sessions, contexts, system maintenance/transfer operations, source-module versioning و platform-specific OCI runtime awareness تقویت شد.